

Unidade IV

7 SISTEMAS DISTRIBUÍDOS

Um sistema distribuído é um conjunto de computadores independentes entre si, podendo até ser de diferentes modelos ou fabricantes, que são ligados através de uma rede de comunicação de dados e que se apresentam aos usuários como um sistema único e coerente (Tanenbaum; Steen, 2007).

Muitas empresas possuem um número significativo de computadores em operação, frequentemente instalados em locais diferentes entre si. Segundo Tanenbaum, Feamster e Wetherall (2021), esses computadores, inicialmente, funcionavam de forma independente dos demais, mas, em um determinado momento, decidiu-se conectá-los para que fosse possível extrair e correlacionar informações sobre toda a empresa, sendo que o objetivo desse compartilhamento de recursos é colocar todos os programas, equipamentos e, especialmente, dados ao alcance de todas as pessoas da empresa, independentemente da localização física do recurso e do usuário.

Nos primórdios da computação, os sistemas eram todos centralizados. Todo o processamento era feito no mainframe. Os terminais eram usados apenas para entrada e saída de dados. Com o tempo, surgiram novas arquiteturas de computação que buscavam descentralizar o processamento, o que deu origem aos primeiros sistemas distribuídos. A ideia, como o próprio termo sugere, era a distribuição do processamento em vez de adotar uma arquitetura centralizada no mainframe.

Atualmente, as empresas estão não apenas participando de redes, como também estão cada vez mais organizadas em redes descentralizadas e operando com sistemas distribuídos. Isso implica em uma maior heterogeneidade no que diz respeito às diferentes aplicações que trabalham em conjunto, como um sistema único. Toda essa heterogeneidade, geralmente, não é visível para os usuários e não é para ser mesmo, pois questões, tais quais "onde" ou "como", são processados os serviços de um sistema distribuído devem ser transparentes para quem está utilizando o sistema. O que importa para o usuário é que o sistema funcione a contento.

Os sistemas distribuídos estão em todo lugar, ou melhor, acessíveis a partir de qualquer lugar. Os componentes interligados em rede podem estar separados por qualquer distância — no mesmo edifício de uma empresa, na mesma cidade, no mesmo estado, no mesmo país ou até em continentes diferentes.

Através dos sistemas distribuídos, os componentes localizados em computadores que estão interligados em rede se comunicam e coordenam suas ações apenas passando mensagens. É comum pensarmos em componentes como sendo apenas servidores e máquinas conectadas em rede, mas quando falamos de componentes, eles podem ser de hardware (servidores) ou de software (aplicações e serviços).

São alguns exemplos de sistemas distribuídos:

- **Internet:** a internet é o maior exemplo de sistema distribuído, pois tem uma extensão muito grande e um alcance global. Ela conecta usuários através de sites e serviços, e-mail, transferência de arquivos, chat etc. Os serviços estão localizados em máquinas dispersas que são interligadas por algum tipo de rede. Diferentes aplicações e serviços se comunicam através da rede, trocando mensagens através de um meio de comunicação e de protocolos, o que permite que aplicações e serviços em plataformas diferentes consigam se comunicar e cooperar.
- **Intranet:** a intranet é uma parte da internet administrada separadamente por uma empresa. Uma intranet é composta de várias redes locais (LANs) interligadas por conexões de backbone. A configuração de rede de uma intranet pode variar desde uma LAN em um único site até um conjunto de LANs interconectadas pertencentes às filiais de uma grande empresa, localizadas inclusive em países diferentes. Uma intranet pode ser conectada à internet através de um roteador que permite aos usuários terem acesso a serviços da internet, que são providos fora da empresa. Também é possível que os usuários de outras intranets tenham acesso aos serviços que outra oferece. A segurança é crucial, pois não deve ser permitido que um usuário de uma empresa concorrente tenha acesso aos projetos e clientes de uma empresa, da mesma forma que um hospital pode não querer que informações dos seus pacientes sejam acessadas por empresas farmacêuticas. A solução envolve não apenas projeto e arquitetura do software, mas também infraestrutura na qual o software será executado.
- **Computação em nuvem:** a computação em nuvem é construída sobre sistemas distribuídos que conectam diversos recursos e dispositivos, tais como computadores, notebooks, smartphones e tablets, além de outros dispositivos IoT que são incorporados em carros, geladeiras, máquinas industriais, smartwatches etc. Os recursos de vários computadores são combinados para criar um sistema virtualizado, que pode ser acessado por usuários em qualquer lugar com conexão à internet. A maioria dos sites e sistemas Web funciona processando as requisições para a nuvem, que então envia as respostas. Serviços em nuvem, como as plataformas Google Cloud, AWS (Amazon Web Services) e Microsoft Azure, utilizam sistemas distribuídos para permitir que os clientes acessem recursos de computação e armazenamento remoto.
- **Aplicativos móveis:** apps que são instalados nos celulares e smartphones, como os aplicativos de compras, redes sociais, serviços de streaming ou mensageiros, frequentemente usam sistemas distribuídos, em que o app é o cliente que se conecta a um servidor remoto para obter dados e enviar informações. Com o avanço das redes sem fio e da miniaturização do hardware, temos componentes interligados por redes sem fio com alta portabilidade e capacidade de se conectar com redes em lugares diferentes. A computação móvel também é chamada computação nômade e permite a execução de tarefas de computação com alta mobilidade.

Os sistemas distribuídos já são uma realidade e estão embutidos nos vários recursos que utilizamos no nosso dia a dia. Por isso, é importante conhecer seus conceitos, suas características e suas arquiteturas.

7.1 Características dos sistemas distribuídos

Uma das características principais dos sistemas distribuídos é que seus componentes estão dispersos geograficamente. Em um sistema distribuído, os elementos principais são chamados de nós ou nodes. Esses nós podem ser tanto máquinas físicas quanto processos virtuais executados em ambientes de nuvem (computação em nuvem ou cloud computing). Cada nó representa um processo de software que conhece outros processos e é capaz de se comunicar diretamente com eles. Essa comunicação acontece por meio do envio de mensagens, que servem para coordenar a execução das tarefas do sistema.



Observação

Edge computing é um modelo complementar aos sistemas distribuídos, no qual o processamento de dados ocorre o mais próximo possível da origem da informação, como sensores IoT ou dispositivos móveis. Isso reduz a latência, desafoga a nuvem e aumenta a eficiência em aplicações que demandam respostas em tempo real, como carros autônomos, fábricas inteligentes e jogos online.

O compartilhamento de recursos é outra importante característica dos sistemas distribuídos. Recursos podem ser compartilhados por diferentes servidores e consumidos por clientes. Serviços podem ser compartilhados através da internet, sendo encapsulados em objetos que podem ser acessados por outros objetos ou por aplicações cliente.

Outro fator marcante dos sistemas distribuídos é sua dinamicidade. Ou seja, os nós podem entrar ou sair do sistema a qualquer momento, sem comprometer o funcionamento geral ou a estrutura da rede.

Existem outras características que surgem como consequência das apresentadas até aqui. De acordo com Coulouris *et al.* (2013), seriam elas:

- **Concorrência:** como os processos são distribuídos pela rede, a execução concorrente de programas é uma característica inerente dos sistemas distribuídos. Um servidor executa um processo enquanto outro processo é executado em outra máquina, sendo que ambos compartilham recursos como banco de dados, largura de banda de rede, discos etc. A disputa por esses recursos compartilhados pelos processos que são executados concorrentemente é uma consequência dos sistemas distribuídos, e também um desafio a ser superado. A coordenação de processos em execução concorrente é uma das formas de tratar a concorrência.
- **Inexistência de relógio global:** em um sistema distribuído, os processos precisam cooperar de forma coordenada através da troca de mensagens. A coordenação considera o compartilhamento do tempo em que os processos são executados. O problema é que existem limites para a precisão com a qual os servidores sincronizam os relógios em uma rede. O fuso horário é um fator complicador na sincronização dos processos. Trabalhando independentemente, cada nó terá sua própria noção de tempo, sincronização e coordenação

em sistemas distribuídos, precisando lidar com o desafio da ausência de um relógio global. Por esse motivo, o conceito de controle de admissão (admission control) é aplicado, com o objetivo de gerenciar a comunicação e o controle de membros para grupos fechados dentro de um cluster de elementos. Assim, mecanismos de autenticação e comunicação entre membros de grupos fechados e abertos podem apresentar problemas de segurança que necessitem ser superados de forma a prosperarem nesse tipo de arquitetura de comunicação.

- **Falhas independentes:** em qualquer sistema existem falhas, e nos sistemas distribuídos não seria diferente. Os projetistas devem considerar as consequências de cada falha. Falhas de rede, de banco de dados, de comunicação ou de qualquer tipo podem acontecer. As falhas na rede são problemáticas pois elas resultam no isolamento dos componentes do sistema distribuído que não podem mais se comunicar e trabalhar juntos. Lentidão na rede também pode afetar os processos em execução. A falha de um componente ou término de um processo em um componente nem sempre é percebida imediatamente pelos demais componentes.

Sistemas distribuídos atuando de forma coerente significa que a sensação proposta de percepção de um único sistema atendendo suas necessidades foi bem-sucedida, algo sustentado pelo princípio de transparência de acesso, formalmente encontrado na computação distribuída.

Em um sistema distribuído adequadamente elaborado, usuários (sejam pessoas ou aplicações) acreditam estar lidando com um sistema único para fins de processamento, quando na verdade existem múltiplos nós computacionais, representados tanto como dispositivos de hardware quanto como processos de software. Operando em conjunto, tais componentes entregam uma visão unificada ao usuário final, sendo a transparência o resultado de que esse usuário sequer note que está lidando com processos e dados dispersos, controlados ao longo de uma rede de computadores distribuída. Por isso, Tanenbaum e Steen (2007, p. 1) definem o sistema distribuído como uma "coleção de computadores independentes que se apresenta a seus usuários como um sistema único e coerente". Em outras palavras, mesmo sendo formado por várias partes autônomas, um sistema distribuído deve oferecer uma experiência integrada e transparente ao usuário final.

Em contrapartida, é importante entender que, a partir do momento em que sistemas distribuídos são compostos por diversos nós espalhados pela rede, que podem falhar a qualquer momento em qualquer ponto, existe uma possibilidade de desencadeamento de comportamentos inesperados, adicionando complexidade ao processo final.

Daí surgem diversos desafios para se implementar e operacionalizar um sistema distribuído. Entre eles, podemos citar:

- **Comunicação entre os componentes:** desafio de como garantir que todos os componentes possam se comunicar, considerando ambientes heterogêneos de rede, servidores, diferentes padrões de arquitetura e protocolos de comunicação.
- **Escalabilidade:** como fazer com que o sistema distribuído suporte o aumento no número de usuários ou requisições.

- **Tolerância a falhas:** como garantir a tolerância a falhas em um sistema com componentes distribuídos em servidores conectados a redes e ambientes remotos.
- **Desempenho:** como assegurar um desempenho aceitável em um sistema distribuído, em que os componentes estão interligados por redes que nem sempre possuem a mesma velocidade e no qual os componentes nem sempre têm a mesma capacidade de processamento, sem que surjam gargalos que comprometam o desempenho.
- **Segurança:** como podemos certificar que somente os clientes válidos têm acesso aos serviços e componentes do sistema.

Vamos ver com mais detalhes esses pontos, pois eles fazem parte do rol de características de um sistema distribuído.

7.1.1 Comunicação e coordenação

A comunicação entre processos por meio de uma rede de computadores é a espinha dorsal de qualquer sistema distribuído. Ela é o que torna possível que diferentes componentes, muitas vezes executados em locais distintos, trabalhem de forma coordenada como se fossem parte de um único sistema. Para que essa comunicação funcione de forma eficiente e confiável, são utilizados protocolos de rede organizados em uma pilha (stack). Cada camada é responsável por uma parte específica do processo de comunicação e tem como objetivo ocultar os detalhes de seu funcionamento das camadas superiores, facilitando a modularidade, o desenvolvimento e a integração dos sistemas.

Nos sistemas distribuídos, a qualidade da comunicação entre os componentes é determinante para o sucesso da aplicação como um todo. Um sistema distribuído pode falhar ou apresentar baixo desempenho se a comunicação entre os processos for mal planejada ou ineficiente. Quando um processo deseja enviar uma mensagem para outro através da rede, essa mensagem percorre as camadas da stack de rede, como vimos na seção 3.5. A comunicação se inicia na camada mais alta (camada de Aplicação) e desce até as camadas mais próximas do hardware, como a camada de Acesso à rede, que efetivamente envia os dados utilizando os pacotes Ethernet através de switches da rede. Os dispositivos envolvidos utilizam seus endereços físicos (MAC addresses) para garantir que a comunicação ocorra entre as máquinas corretas na rede.

Durante esse percurso, os dados passam também pela camada Internet, na qual ocorre o roteamento de pacotes IP. Nessa etapa, os pacotes são encaminhados do endereço IP de origem até o endereço IP de destino.

Na camada de Transporte, o protocolo TCP é o responsável por garantir que os dados sejam entregues de forma ordenada e sem perdas. O TCP estabelece uma conexão entre dois pontos, cada um identificado por um endereço IP e uma porta, e controla a entrega das mensagens. Para aumentar a segurança, é comum utilizar o protocolo TLS (Transport Layer Security) junto com o TCP. O TLS criptografa os dados transmitidos, garantindo a confidencialidade e a integridade das mensagens, criando uma camada de comunicação segura sobre o TCP.

Outro elemento essencial da comunicação na internet é o protocolo DNS (Domain Name System). Ele atua como uma espécie de catálogo telefônico, traduzindo nomes de host legíveis (como `www.exemplo.com`) para endereços IP. O DNS opera de forma distribuída, hierárquica e eventualmente consistente, permitindo que os usuários encontrem os serviços desejados mesmo em uma estrutura altamente descentralizada.

À medida que as aplicações modernas são decompostas em serviços menores e mais especializados, esses serviços frequentemente se comunicam trocando mensagens. Esse tipo de comunicação geralmente ocorre de maneira assíncrona, ou seja, os processos que enviam e recebem mensagens não precisam estar ativos ao mesmo tempo. Essa comunicação desacoplada é viabilizada por canais de mensagens (message channels), e os sistemas distribuídos adotam diferentes padrões para troca de mensagens, como o envio unilateral (one-way messaging), a requisição com resposta (request-response messaging) e a difusão para múltiplos receptores (broadcast messaging). A escolha entre esses padrões envolve sempre um equilíbrio entre desempenho, confiabilidade e complexidade.

Para garantir que as mensagens sejam processadas em ordem, muitos sistemas utilizam intermediários especializados chamados message brokers. Um exemplo bastante conhecido é o Apache Kafka, que organiza os canais de mensagens (chamados topics) em subcanais (partitions). Cada partição pode ser consumida por um único processo consumidor, que é o responsável por manter a consistência do processamento. Cada message broker pode adotar comportamentos específicos, mas, independentemente da implementação, esses componentes desempenham um papel essencial na organização, entrega e monitoramento das mensagens em sistemas distribuídos.

Além da comunicação eficiente, a construção de sistemas distribuídos também exige coordenação entre os processos. Isso inclui lidar com falhas de rede ou de hardware, que são inevitáveis em ambientes distribuídos. Para isso, são utilizados diferentes modelos e tipos de links de comunicação. Por exemplo, temos o fair-loss link, que assume que mensagens podem ser perdidas; o reliable link, que tenta garantir a entrega; e o authenticated reliable link, que adiciona verificação de identidade e integridade das mensagens. Esses modelos são abstrações que nos ajudam a projetar algoritmos para detectar e recuperar falhas de forma mais eficaz.

Dentro dessa perspectiva, é comum o uso de algoritmos como arbitrary-fault, crash-recovery e crash-stop para lidar com falhas de processos. Esses algoritmos operam em conjunto com modelos de tempo – synchronous, asynchronous e partially synchronous –, que descrevem a forma como o tempo é percebido e utilizado para a comunicação e coordenação entre os nós.

Quando falamos em envio de mensagens, também é importante entender as garantias de entrega. As mais comuns são:

- **At-most-once:** a mensagem é entregue no máximo uma vez, podendo não ser entregue.
- **At-least-once:** a mensagem é entregue pelo menos uma vez, podendo ser duplicada.
- **Exactly-once:** a mensagem é entregue uma única vez, sem perdas ou duplicações.

No contexto dos dados, quando os sistemas distribuídos precisam alterar o estado da aplicação, frequentemente adotam modelos baseados em líderes e réplicas. Nesse modelo, um dos nós atua como líder e é o único autorizado a receber e processar solicitações de escrita. Já as réplicas podem servir aos consumidores com leituras dos dados. Esse modelo melhora a escalabilidade e a disponibilidade, ao mesmo tempo que permite manter a consistência entre os nós, embora, por causa dos atrasos de replicação, dois consumidores diferentes possam ter visões distintas do estado dos dados em um dado momento.

Restringir as alterações de estado ao líder evita a sobrecarga de coordenação entre todos os nós, mas também impõe um importante compromisso de desempenho. Assim, enquanto apenas o líder processa escritas, as réplicas podem atender requisições de leitura, o que equilibra a carga do sistema.

A escolha do modelo de consistência adotado – seja ele forte (strong), sequencial (sequential) ou eventual (eventual) – tem implicações diretas sobre a experiência do usuário e a confiabilidade do sistema. O modelo de consistência define como e quando as atualizações nos dados são refletidas para os diversos processos que compõem o sistema. No modelo strong consistency, todos os nós veem as alterações ao mesmo tempo. No sequential consistency, as operações são vistas na mesma ordem, mas não necessariamente ao mesmo tempo. Já na eventual consistency, os nós acabam convergindo para o mesmo estado, mas isso pode levar algum tempo.

Em um ambiente distribuído, é muito difícil garantir simultaneamente consistência, disponibilidade e tolerância a particionamentos de rede. Portanto, sempre será necessário sacrificar uma dessas propriedades em detrimento das outras, de acordo com o contexto da aplicação.

7.1.2 Escalabilidade

A escalabilidade se destaca como uma das maiores vantagens dos sistemas distribuídos. Ela permite que uma aplicação aumente sua capacidade de processamento à medida que a demanda cresce, e pode ser realizada de duas formas: verticalmente (scaling up), por meio do fortalecimento de um único nó com mais recursos computacionais; ou horizontalmente (scaling out), com a adição de novos nós ao sistema.

A escalabilidade horizontal, em especial, é a abordagem predominante nos sistemas modernos, por permitir que a carga de trabalho seja dividida entre múltiplos nós, garantindo mais eficiência e resposta rápida às exigências do ambiente.

Existem três estratégias principais para alcançar a escalabilidade horizontal: duplicação, decomposição funcional e particionamento.

A duplicação é, sem dúvida, a técnica mais comum. Ela consiste em replicar instâncias da aplicação e usar mecanismos de balanceamento de carga (load balancing) para distribuir as requisições entre essas cópias, garantindo uma resposta rápida mesmo em momentos de pico. Essa abordagem, por ser simples de implementar e de baixo custo, é amplamente utilizada para manter o desempenho esperado diante do aumento de usuários ou transações.

Por outro lado, a decomposição funcional visa a dividir a aplicação em partes menores, especializadas e independentes. Cada uma dessas partes pode ser desenvolvida, implantada e escalada separadamente, o que traz flexibilidade e robustez ao sistema. Essa estratégia dá origem a um estilo arquitetural muito comum em sistemas distribuídos contemporâneos: a arquitetura de microsserviços. Nessa abordagem, cada microsserviço é responsável por uma função específica do sistema, comunica-se com os demais por meio de mensagens ou APIs e pode evoluir de forma independente, sem afetar o restante do ecossistema. Veremos mais sobre microsserviços ainda nesta unidade.

A terceira estratégia, o particionamento, também contribui para a escalabilidade horizontal, dividindo os dados ou funcionalidades em segmentos menores e alocando-os entre diferentes nós. Essa divisão permite que operações sejam realizadas em paralelo, acelerando o processamento geral da aplicação.

À medida que uma aplicação distribuída cresce horizontalmente para atender a cargas de trabalho maiores, também aumenta a probabilidade de ocorrência de falhas. Quanto mais componentes estão envolvidos na execução de tarefas, maior a complexidade do sistema e, consequentemente, maiores os riscos de interrupções. Por esse motivo, a alta disponibilidade se torna um requisito essencial em ambientes distribuídos. Para alcançá-la, não basta apenas escalar o sistema; é necessário também adotar práticas de scale-out de forma planejada e eficaz.

Além disso, é fundamental implementar mecanismos de resiliência que permitam ao sistema continuar operando mesmo diante de falhas, bem como técnicas de self-healing, ou seja, a capacidade de se autorrecuperar automaticamente. Esses mecanismos detectam falhas, isolam os componentes comprometidos e restabelecem o funcionamento normal sem necessidade de intervenção humana. Combinando essas estratégias, é possível construir sistemas distribuídos verdadeiramente robustos, preparados para lidar com grandes volumes de dados, tráfego intenso e ambientes imprevisíveis, mantendo sempre o desempenho, a confiabilidade e a experiência do usuário em primeiro plano.

7.1.3 Tolerância a falhas

Em sistemas distribuídos, falhas podem ocorrer por diferentes motivos, e é essencial compreendê-los para projetar aplicações mais resilientes.

Entre os tipos mais comuns de falhas estão: o ponto único de falha (single point of failure), que compromete todo o sistema caso um único componente falhe; redes instáveis ou imprevisíveis (unreliable network), que causam perda ou atraso na comunicação entre componentes; processos lentos ou sobrecarregados (slow processes), que prejudicam a fluidez das operações; cargas inesperadas (unexpected load), que podem levar ao colapso de recursos; e falhas em cascata (cascading failures), nas quais o erro em um componente se propaga para outros. Diante desses riscos, o uso de mecanismos de resiliência e estratégias de retentativas automáticas é essencial para manter a estabilidade e o desempenho da aplicação.

Esses mecanismos de proteção contra falhas podem ser classificados em dois grupos principais: downstream e upstream. Os problemas de tipo downstream ocorrem quando um serviço consome outros serviços e precisa lidar com a instabilidade desses serviços externos, que podem comprometer

sua performance. Já as falhas upstream ocorrem quando um serviço é requisitado em excesso por clientes ou consumidores e não consegue responder adequadamente a todas as solicitações.

Para lidar com falhas do tipo downstream, é fundamental que o sistema evite sobrecarregar os recursos disponíveis. Isso é particularmente importante em razão da limitação de conexões simultâneas que um servidor pode manter em suas portas. Quando ocorre uma sobrecarga de requisições, os servidores podem adotar técnicas de alívio de carga, recusando temporariamente novas conexões ao retornar um código HTTP 503 (Service Unavailable), indicando que o serviço está temporariamente indisponível.

Por outro lado, ao tratar de falhas do tipo upstream, outras abordagens são aplicadas para garantir que o servidor não fique sobrecarregado a ponto de comprometer a estabilidade do sistema como um todo. Técnicas como load leveling, que visa a redistribuir a carga ao longo do tempo, e rate limiting, que limita a quantidade de requisições recebidas em um intervalo de tempo, ajudam a proteger os recursos do servidor. Essas medidas garantem que mesmo sob alta demanda, o serviço consiga manter baixa latência e alto throughput, entregando um desempenho aceitável aos seus usuários.



Lembrete

A rede de computadores é uma infraestrutura fundamental para qualquer sistema distribuído. Devido a isso, falhas em qualquer camada da arquitetura de rede podem ter um impacto significativo no funcionamento do sistema como um todo.

Um aspecto fundamental para a manutenção da resiliência dos sistemas é a verificação contínua da saúde dos componentes. Através da execução de health checks, é possível monitorar recursos essenciais como uso de CPU, memória e disco, avaliando continuamente o estado dos serviços em execução. Esses indicadores permitem decisões rápidas e informadas, como a substituição ou o reinício automático de componentes comprometidos.

Além disso, para garantir que falhas não se espalhem por todo o sistema, é necessário isolar os componentes afetados. Esse isolamento, conhecido como o padrão bulkhead, funciona de maneira análoga a compartimentos estanques em navios, que impedem que uma falha em um módulo comprometa o restante do sistema. Outro padrão amplamente adotado é o circuit breaker, que monitora o comportamento de chamadas entre componentes e, ao detectar um número elevado de falhas, interrompe temporariamente as tentativas de comunicação, permitindo que o serviço problemático se recupere antes de retomar as operações normais.

Essas estratégias, quando aplicadas de maneira adequada, aumentam significativamente a confiabilidade dos sistemas distribuídos, permitindo que os componentes se comportem de forma mais previsível, mesmo em cenários de falhas parciais ou instabilidade. O resultado é um sistema mais robusto, com maior qualidade percebida pelos usuários finais e menor risco de interrupções críticas.

7.1.4 Desempenho

Os sistemas distribuídos devem ser capazes de manter um desempenho elevado, mesmo em situações em que um único nó de execução seja fisicamente incapaz de lidar com a carga de trabalho.

O desempenho é um dos pilares fundamentais no desenvolvimento e operação de sistemas distribuídos. Ao lado da escalabilidade, da disponibilidade e da tolerância a falhas, ele determina diretamente a experiência do usuário final e a eficiência geral da aplicação. Em um ambiente distribuído, o desempenho não se resume apenas à velocidade de execução de uma tarefa isolada, mas envolve uma complexa interação entre diversos componentes, espalhados por diferentes máquinas, redes e regiões geográficas. A forma como esses elementos se comunicam, processam informações e respondem às requisições define a capacidade do sistema de atender a um grande número de usuários de forma rápida e eficaz (Coulouris *et al.*, 2013).

O tempo de resposta de uma aplicação distribuída é um reflexo direto da sua latência e da sua capacidade de throughput. Latência é o tempo que uma operação leva desde o momento em que é requisitada até a entrega do resultado, enquanto throughput representa o volume de operações que o sistema consegue processar em um determinado intervalo de tempo. Esses dois conceitos são interdependentes: muitas vezes, melhorar o throughput implica lidar com aumentos na latência e vice-versa. Encontrar o equilíbrio adequado entre esses fatores é essencial para manter o desempenho ideal em aplicações críticas, como sistemas bancários, plataformas de streaming ou redes sociais em tempo real.

Outro ponto essencial a ser considerado é o impacto da distribuição física dos componentes do sistema. Em arquiteturas distribuídas, os serviços e bancos de dados podem estar executando em diferentes servidores, localizados em datacenters distintos. Cada chamada entre serviços, portanto, pode envolver saltos de rede que adicionam latência à operação. Por isso, otimizar o desempenho em sistemas distribuídos passa obrigatoriamente por decisões de arquitetura que levem em consideração a proximidade dos dados em relação aos processos que os utilizam. Técnicas como o cache local, replicação de dados e estratégias de edge computing (computação na ponta) são frequentemente empregadas para reduzir os tempos de resposta e reduzir o tráfego de rede. No edge computing, o processamento e o armazenamento de dados são realizados o mais próximo possível da fonte de dados, o que minimiza a latência e reduz a necessidade de transferir grandes volumes de dados para a nuvem.

Além disso, o desempenho está diretamente ligado à eficiência do código e dos algoritmos implementados. Um serviço mal otimizado, mesmo que executado em uma infraestrutura poderosa, pode se tornar um gargalo em um ambiente distribuído. Por esse motivo, é fundamental que cada componente da aplicação seja analisado e testado continuamente em relação ao seu consumo de CPU, memória e operações de disco. A identificação e eliminação de pontos de congestionamento ou lentidão, conhecidos como bottlenecks, garante que o sistema como um todo mantenha uma performance estável, mesmo quando submetido a picos de uso ou aumentos inesperados na carga de trabalho.

Outro aspecto que influencia o desempenho é o modelo de comunicação entre os serviços. Sistemas que utilizam chamadas síncronas entre serviços, por exemplo, estão sujeitos a tempos de espera mais

altos, especialmente se um dos serviços estiver com problemas de latência. Por isso, muitos arquitetos optam por modelos de comunicação assíncronos, utilizando filas e brokers de mensagens para desacoplar os serviços e evitar bloqueios. Essa abordagem permite que os processos continuem fluindo mesmo diante de lentidão em componentes específicos, melhorando o desempenho geral da aplicação.

Por fim, o desempenho precisa ser constantemente monitorado e ajustado ao longo do ciclo de vida do sistema. Métricas como tempo médio de resposta, uso de recursos, taxa de erro e capacidade de throughput são indispensáveis para entender o comportamento real da aplicação em produção. Com essas informações em mãos, é possível realizar um ajuste dinâmico dos serviços e dos limites de recursos, reconfigurar balanceadores de carga e escalar os componentes de maneira precisa e eficiente.

Garantir um bom desempenho em sistemas distribuídos é, portanto, um esforço contínuo, que exige atenção aos detalhes, boas práticas de desenvolvimento e uma infraestrutura bem planejada. Não se trata apenas de fazer o sistema funcionar, mas de fazê-lo responder com rapidez, fluidez e consistência, mesmo nos momentos de maior exigência. Quando o desempenho é tratado como prioridade desde as fases iniciais do projeto, torna-se possível entregar soluções robustas, eficientes e capazes de oferecer uma excelente experiência para milhões de usuários ao redor do mundo.

7.1.5 Segurança

Os componentes que formam um sistema distribuído precisam atender a requisitos rigorosos para funcionarem de maneira eficaz em ambientes complexos e de alta demanda. Entre essas exigências, vimos que estão a comunicação, a alta escalabilidade, a tolerância a falhas e o desempenho. Mas um pré-requisito é necessário para que todos os outros consigam ser atendidos: a segurança.

Em sistemas distribuídos, nos quais diferentes componentes operam de forma autônoma e cooperativa em múltiplas máquinas, redes e regiões, a segurança se torna um fator crítico. A natureza descentralizada dessas arquiteturas, embora traga benefícios significativos como escalabilidade, resiliência e performance, também amplia a superfície de ataque e exige cuidados redobrados para garantir a integridade, a confidencialidade e a disponibilidade das informações que trafegam entre os diversos pontos do sistema. A segurança, nesse contexto, não é um atributo isolado, mas sim uma responsabilidade transversal que permeia todas as camadas da arquitetura, desde a infraestrutura até o código da aplicação.



Observação

A superfície de ataque, em segurança cibernética, refere-se a todos os pontos onde um atacante pode tentar comprometer um sistema ou rede. É o conjunto de vulnerabilidades e pontos de entrada que podem ser explorados por cibercriminosos para obter acesso não autorizado.

Proteger um sistema distribuído é um desafio complexo, especialmente porque os componentes se comunicam constantemente entre si, muitas vezes através de redes públicas ou não confiáveis. Cada

troca de mensagens entre serviços representa uma potencial vulnerabilidade. Por isso, é fundamental que todas as comunicações sejam realizadas de forma segura, por meio de canais criptografados, como os fornecidos pelo protocolo TLS. Isso garante que os dados transmitidos não possam ser interceptados, adulterados ou lidos por agentes maliciosos durante o tráfego em rede. Além da criptografia em trânsito, é igualmente importante proteger os dados em repouso, utilizando criptografia nos bancos de dados, sistemas de arquivos e backups, assegurando que as informações estejam protegidas mesmo que haja acesso físico aos dispositivos de armazenamento.

Outro aspecto crucial da segurança em sistemas distribuídos é o controle de identidade e acesso. Como múltiplos serviços e usuários interagem com o sistema, é necessário garantir que apenas entidades autorizadas possam executar determinadas ações. Mecanismos como autenticação forte, baseada em tokens, certificados ou chaves criptográficas, ajudam a garantir que cada requisição provenha de uma fonte legítima. Complementando a autenticação, a autorização define os níveis de permissão de cada entidade, evitando que um serviço ou usuário acesse dados ou funções que não lhe dizem respeito. O princípio do menor privilégio, amplamente adotado em ambientes seguros, recomenda que cada componente do sistema tenha apenas as permissões estritamente necessárias para seu funcionamento.

Além da autenticação e da autorização, a rastreabilidade das ações realizadas no sistema é uma peça-chave da segurança. Isso significa manter registros detalhados de eventos, acessos e transações, através de mecanismos de auditoria. Logs bem estruturados permitem identificar comportamentos suspeitos, diagnosticar incidentes e investigar possíveis violações de segurança. Em sistemas distribuídos, essa rastreabilidade deve ser coordenada de forma global, integrando informações de diferentes serviços e regiões em um único ponto de visibilidade, geralmente por meio de ferramentas de observabilidade e monitoramento de segurança.

A segurança também está diretamente relacionada à proteção contra ataques que visam explorar vulnerabilidades conhecidas, como injeção de código, execução remota ou negação de serviço (DoS e DDoS). Para mitigar esses riscos, é fundamental adotar boas práticas de desenvolvimento seguro, como validação de entrada, gerenciamento seguro de dependências e atualizações contínuas de segurança. Em sistemas distribuídos, essas práticas devem ser automatizadas dentro de pipelines de integração e entrega contínua, de modo que cada componente seja construído, testado e implantado com medidas de proteção incorporadas desde o início.

Outro ponto essencial diz respeito à resiliência contra acessos indevidos, especialmente em cenários nos quais múltiplas organizações ou equipes compartilham a mesma infraestrutura. A segmentação de ambientes, o isolamento de redes e o uso de máquinas virtuais ajudam a impedir que um comprometimento em um ponto afete todo o sistema. Adicionalmente, técnicas como rotação periódica de segredos, autenticação mútua entre serviços e uso de cofres seguros para armazenamento de credenciais evitam a exposição acidental de dados sensíveis.

Por fim, vale lembrar que a segurança em sistemas distribuídos não é um estado permanente, mas um processo contínuo. Novas vulnerabilidades são descobertas constantemente, e os ataques evoluem de forma rápida e imprevisível. Por isso, é imprescindível que haja uma cultura de segurança enraizada na equipe, com políticas claras, treinamentos frequentes e auditorias regulares. A segurança precisa ser

considerada desde os estágios iniciais do projeto até a operação em produção, sendo tratada como uma responsabilidade coletiva, e não apenas da área de infraestrutura ou da equipe de segurança.

Ao investir em segurança de forma proativa e estruturada, os sistemas distribuídos tornam-se mais confiáveis, protegendo não apenas os dados que manipulam, mas também a reputação das organizações que os operam. Em um mundo cada vez mais conectado e digital, a segurança deixa de ser um diferencial e passa a ser um requisito indispensável para o sucesso e a longevidade de qualquer sistema distribuído moderno.

7.2 Comparação com sistemas centralizados e paralelos

Nos sistemas centralizados, existe um único componente, que é o próprio sistema centralizado. No entanto, em sistemas distribuídos é um pouco diferente, pois temos diversos componentes espalhados, ou distribuídos por diferentes servidores, que podem ser heterogêneos, com diferentes sistemas operacionais, de diferentes fabricantes, com diferentes capacidades de processamento.

Um sistema centralizado é um sistema no qual um indivíduo, um grupo de pessoas ou uma corporação detém o total controle das funcionalidades do sistema. Redes sociais como Facebook, X (antigo Twitter) etc. são exemplos de sistemas centralizados.

Os usuários finais não têm nenhum papel final no design da arquitetura ou da plataforma, nem controle sobre a disponibilidade de recursos ou funcionalidade das aplicações. Os usuários não decidem como os sistemas devem operar, além de não serem donos dos dados que eles mesmos compartilham na plataforma. Inclusive, entidades corporativas possuem os direitos de modificar ou deletar os dados dos usuários sem nenhuma permissão.

Um sistema centralizado tem seus riscos, como, por exemplo, um único ponto de falha. Se a corporação fechar, todos os dados de quem utilizava a plataforma são perdidos – foi o que aconteceu com o Orkut, por exemplo.

Costuma-se comparar os sistemas centralizados a sistemas monolíticos, que vem do termo **monólito**. Monólito significa "obra construída em uma só pedra", por isso é utilizado para definir a arquitetura de alguns sistemas. Refere-se à forma de desenvolver um sistema, programa ou aplicação em que todas as funcionalidades e códigos estão em um único processo. Essas diversas funcionalidades estão em um mesmo código-fonte e, em sua execução, compartilham recursos da mesma máquina, seja o processador, a memória, os bancos de dados ou os arquivos.

Em termos menos formais, programadores ou desenvolvedores de sistemas podem já ter feito o uso da palavra **monólito** para se referir a algum sistema grande, que tem apenas um código-fonte. Como o sistema está inteiro em um único bloco, seu desenvolvimento é mais ágil, se comparado com outras arquiteturas, sendo possível desenvolver uma aplicação em menos tempo e com menor complexidade inicial.

A diferença entre sistemas centralizados e monolíticos está na intenção que essas palavras representam. Sistemas centralizados representam controle, ou seja, falam mais sobre o lugar em que

o controle dos dados está situado, enquanto os sistemas monolíticos falam sobre o lugar em que o sistema em si está situado. Ou seja, não chegam a ser conceitos opostos, mas também não representam ideias iguais. Podemos dizer que são apenas propostas diferentes.

Se quisermos achar um termo que represente exatamente o mesmo intuito dos sistemas centralizados para representar o seu oposto, temos os sistemas descentralizados ou distribuídos.

Os sistemas descentralizados ou distribuídos geralmente possuem seu controle nos usuários finais. O sistema Torrent, utilizado para compartilhar e baixar arquivos pela internet, é um exemplo desse tipo de sistema. Ele se diferencia do download tradicional, que depende de um servidor central, ao permitir que arquivos sejam baixados em partes de vários usuários da rede, tornando o processo mais eficiente. Quanto mais pontos forem responsáveis pela disponibilidade do conteúdo na rede, ou quanto mais pessoas estiverem conectadas aos dados, maior é a velocidade de download e melhor é o tempo de acesso.

Além disso, sistemas descentralizados não têm um ponto único de falha como os sistemas centralizados. Se metade dos nós caírem, ou seja, usuários ou participantes dessa aplicação ficarem inativos, a rede como um todo ainda continuará ativa. Além disso, há zero possibilidade de uma rede ficar caída ou os arquivos serem totalmente apagados quando temos tantas conexões e fontes de informação pelo mundo todo.

Sistemas distribuídos fazem parte de um grupo maior da computação conhecido como computação paralela. Esse grupo inclui diversas formas de organização e execução de tarefas que envolvem múltiplos componentes trabalhando ao mesmo tempo. A classificação dessas arquiteturas considera, entre outros fatores, o grau de paralelismo que o hardware é capaz de oferecer e a distância física ou lógica entre os processos que compõem uma aplicação.

Além dos sistemas distribuídos propriamente ditos, esse conjunto inclui outras arquiteturas importantes como:

- **Cluster computing (computação em cluster):** vários computadores conectados em rede, funcionando como se fossem um só.
- **Multi-core computing (computação multi-core):** vários núcleos de processamento dentro de um mesmo chip.
- **Symmetric multiprocessing – SMP (multiprocessamento simétrico):** múltiplos processadores compartilhando a mesma memória.
- **Grid computing (computação em grade):** sistemas geograficamente dispersos que compartilham recursos de forma coordenada.

Em computação distribuída, os nós, ou elementos computacionais, caracterizam-se como um processo de software, equipado com uma lista de outros processos, capazes de enviar mensagens

diretamente uns aos outros, estimulando-os a executar e concluir as operações de negócio descritas pelo sistema.

Nos sistemas distribuídos, os componentes colaboram para realizar uma tarefa predeterminada, comunicando-se através do envio de mensagens, podendo referenciar máquinas físicas ou processos de software rodando em cloud.

7.3 Arquitetura de sistemas distribuídos

A arquitetura de sistemas distribuídos pode diferir de acordo com a perspectiva observada. Do ponto de vista físico, um sistema distribuído não passa de um grupo de máquinas comunicando-se através de uma rede. Em runtime, um sistema pode ser composto por processos comunicando-se via HTTP, rodando em um sistema operacional. Já sob ângulo de implementação, um sistema distribuído pode ser descrito como um conjunto de componentes, com baixo ou nenhum acoplamento, publicados e escalando individualmente cada qual com certo nível de autonomia durante a execução de um processo.

Nodes computacionais autônomos precisam colaborar para efetivamente realizar seu trabalho. Na prática, sistemas distribuídos são frequentemente organizados como overlay networks, a exemplo das redes P2P (ponto a ponto). Independentemente do tipo, redes P2P necessitam de esforço especial para a organização e gerenciamento dos nodes, sendo uma das partes mais complexas quando nos referimos a sistemas distribuídos.

Todo sistema, por mais simples que seja, é construído sobre uma arquitetura. No entanto, nem todas as arquiteturas são igualmente eficazes. Existem arquiteturas bem projetadas, que antecipam e resolvem problemas ao longo do ciclo de vida do sistema, sejam eles técnicos ou relacionados ao negócio, e existem aquelas que falham nesse aspecto. Uma arquitetura sólida é aquela que considera, desde o início, as dificuldades que poderão surgir no futuro e busca soluções para elas.

Entre essas decisões fundamentais está a escolha de tornar ou não o sistema distribuído. Essa escolha faz parte do planejamento arquitetural e traz consigo uma série de implicações que devem ser cuidadosamente analisadas. Optar por uma arquitetura distribuída é mais do que uma questão técnica: é uma decisão estratégica, que exige a compreensão dos desafios próprios desse modelo.

Como já discutimos, a maneira como os componentes de um sistema distribuído são organizados e interagem entre si influencia diretamente as propriedades do sistema, como desempenho, escalabilidade, resiliência e segurança. Em sistemas distribuídos, porém, é importante considerar não apenas a disposição física dos componentes, mas também a forma como eles se estruturam logicamente e se comunicam entre si. Esse aspecto é o que chamamos, de maneira didática, de arquitetura de software.

Quando um sistema distribuído adota uma determinada configuração lógica para organizar seus componentes, estamos diante de um estilo arquitetural. Esses estilos representam padrões recorrentes que ajudam a guiar decisões de projeto e a lidar com complexidade. Vale destacar que esses estilos não são mutuamente exclusivos, pois é bastante comum que uma aplicação combine

múltiplos estilos arquiteturais, aproveitando o que cada um tem de melhor para atender às suas necessidades específicas.

A seguir, exploraremos alguns dos estilos arquiteturais mais frequentemente utilizados no desenvolvimento de sistemas distribuídos, de acordo com Tanenbaum e Steen (2007):

- **Arquitetura em Camadas:** nesse modelo, os componentes do sistema são organizados em camadas hierárquicas, cada uma com uma função bem definida. A principal característica desse estilo é o baixo acoplamento entre as camadas: uma camada só se comunica diretamente com a imediatamente inferior, que oferece seus serviços por meio de uma interface bem especificada. Isso favorece a modularidade, facilita a manutenção e permite maior reutilização de código. Um exemplo bastante conhecido é a arquitetura de três camadas (veja a figura a seguir), frequentemente adotada em aplicações corporativas. Nesse modelo, o sistema é estruturado em três camadas: a camada de apresentação, responsável pela interface com o usuário e pela exibição das informações; a camada de negócio, na qual são implementadas as regras e a lógica principal da aplicação; e a camada de dados, encarregada do armazenamento, acesso e manipulação das informações persistentes.

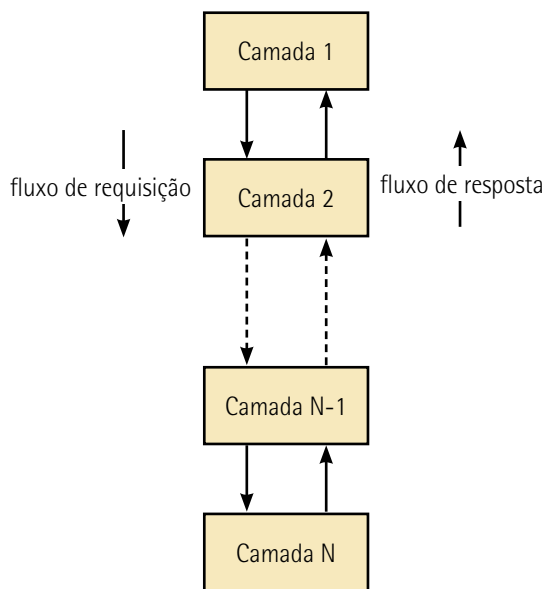


Figura 66 – Arquitetura em Camadas

Fonte: Tanenbaum e Steen (2007, p. 22).

- **Arquitetura Baseada em Objetos:** também conhecida como Arquitetura Orientada a Serviços, esse estilo é mais flexível e menos prescritivo em termos de estrutura. Os componentes do sistema são tratados como objetos autônomos que interagem entre si por meio de chamadas de procedimento remoto (RPC – remote procedure call). A figura a seguir ilustra essa abordagem que promove o encapsulamento e permite a construção de sistemas compostos por módulos que se comportam como serviços reutilizáveis e distribuídos.

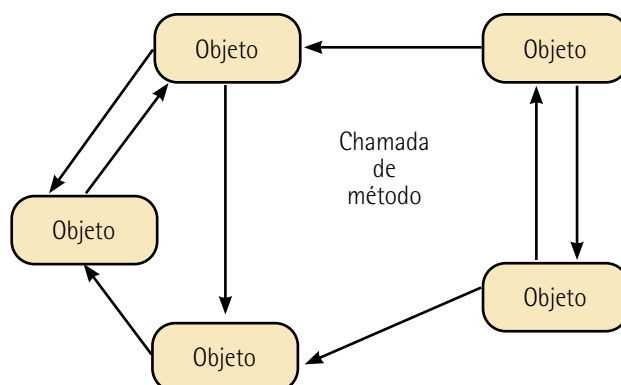


Figura 67 – Arquitetura Baseada em Objeto

Fonte: Tanenbaum e Steen (2007, p. 22).

- **Arquitetura Centrada em Dados:** nesse modelo, a comunicação entre os diversos componentes do sistema ocorre por meio de um repositório de dados compartilhado. Os elementos da aplicação acessam esse espaço comum para ler e gravar informações, o que estabelece uma forma indireta de interação entre eles. A Web é um exemplo prático desse conceito, pois é estruturada em torno de documentos de hipermídia que são acessados, manipulados e transferidos utilizando o protocolo HTTP e seus métodos padrão, como GET, POST, PUT e DELETE. Um exemplo concreto de tecnologia que se alinha a esse estilo arquitetural é o Amazon S3, um serviço de armazenamento em nuvem voltado para a manipulação de grandes volumes de dados por diferentes sistemas. A figura a seguir mostra como funciona a arquitetura Centrada em Dados e a comunicação entre os componentes.

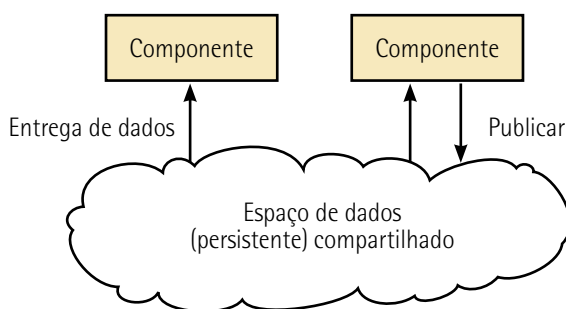


Figura 68 – Arquitetura Centrada em Dados

Fonte: Tanenbaum e Steen (2007, p. 22).

- **Arquitetura Baseada em Eventos:** nesse estilo arquitetural, os componentes do sistema comunicam-se por meio de eventos, alterações de estado que são relevantes dentro do domínio da aplicação. Quando um evento ocorre, ele é encapsulado em uma mensagem e publicado em um canal de comunicação, sem que o sistema emissor (ou produtor) precise conhecer os sistemas que irão reagir a esse evento. Os sistemas que desejam receber notificações sobre determinados tipos de eventos registram seu interesse junto ao canal. Esses sistemas são chamados de subscritores.

Em muitos casos, um mesmo componente pode atuar tanto como produtor quanto como subscritor. A grande força dessa abordagem está no baixo acoplamento entre os elementos do sistema distribuído, o que favorece a escalabilidade e a adaptabilidade em ambientes dinâmicos e complexos. A figura a seguir ilustra esse tipo de arquitetura.

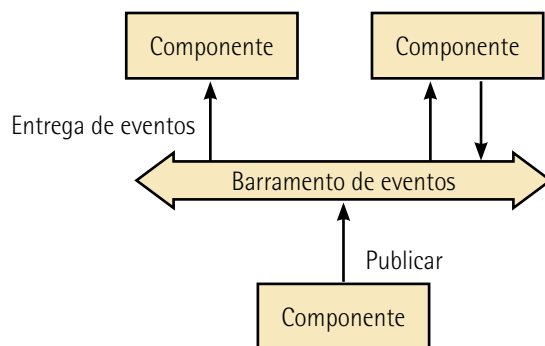


Figura 69 – Arquitetura Baseada em Eventos

Fonte: Tanenbaum e Steen (2007, p. 22).

Encontramos sistemas distribuídos de todos os tamanhos e formas. Atualmente, a maior parte dos sistemas em larga escala são formados por serviços trabalhando em conjunto para entregar funções de negócios.

O conceito de serviços em aplicações não é novo. Assim como os componentes, os serviços são considerados blocos independentes que, juntos, formam o ambiente da aplicação. No entanto, ao contrário dos componentes tradicionais, os serviços possuem características específicas que os tornam adequados para compor uma SOA (Arquitetura Orientada a Serviços).

Uma das principais características dos serviços é a sua autonomia: cada serviço é responsável por um domínio específico, geralmente relacionado a uma função de negócio bem definida ou a um grupo de funcionalidades correlatas. Esse modelo de projeto favorece a criação de unidades isoladas de lógica de negócio, que se comunicam de forma fracamente acoplada. Isso é possível graças à definição de padrões de comunicação bem estabelecidos, como APIs ou mensagens padronizadas.

Como resultado, cada serviço pode ser implementado de forma independente, sem a necessidade de seguir uma plataforma, linguagem ou tecnologia específica, o que dá flexibilidade e escalabilidade à arquitetura do sistema. Neste contexto, um serviço trata da abstração de uma capacidade de negócio consistente em torno de um conjunto de regras bem definidas, podendo expor interfaces para comunicação interna ou externamente.

Existem diversos níveis de abstração pertencentes à jornada de implementação de componentes com natureza distribuída. Entender seus fundamentos técnicos é o principal desafio da arquitetura para componentes distribuídos. Vamos estudar as diferentes maneiras existentes para a decomposição de um bloco de software em partes menores, descobrindo seus relacionamentos e responsabilidades. Essas partes são chamadas de web services, que permitem que diferentes aplicações e sistemas interajam entre si.

7.4 Web services

Um web service, ou serviço web, é uma função abstrata que deve ser implementada por um agente concreto. Ou seja, o web service é uma ideia, um conceito, que define o que deve ser feito – por exemplo, disponibilizar informações de um banco de dados, processar pagamentos ou autenticar usuários.

Ele descreve funcionalidades acessíveis pela rede, normalmente pela internet. Mas, por si só, ele é só uma descrição do serviço, não uma coisa física.

O agente concreto é a aplicação real que realiza o serviço de verdade. É o programa ou sistema que, de fato, responde a pedidos (requisições) feitos ao web service. Esse agente pode ser, por exemplo, um servidor com código em Java, Python, C#, ou qualquer outra linguagem que, executando um programa, atenda os pedidos de clientes via web.

Resumidamente, o agente que aciona o web service é o componente concreto de software ou hardware que envia e recebe mensagens, enquanto o web service é o recurso caracterizado pelo conjunto abstrato de funcionalidades fornecido.

Para ilustrar essa distinção, um determinado web service implementado pode usar um agente em um dia, escrito em uma linguagem de programação qualquer, e usar um agente diferente no dia seguinte, escrito em uma linguagem de programação diferente, mas exercendo a mesma funcionalidade. Embora o agente possa ter mudado, o web service permanece o mesmo. Algumas das principais características dos web services vêm da tríade básica de descrição do serviço, descoberta do serviço e entrega do serviço (W3C, 2004):

- **Service Description (Descrição do Serviço):** cada web service está associado a uma descrição que informa o que o serviço faz e como chamar suas funções. Uma linguagem padrão universal permite a descrição do serviço de forma que ele possa ser compreendido tanto por humanos quanto por máquinas. A linguagem de programação também descreve como esse serviço pode ser contratado.
- **Service Discovering (Descoberta de Serviço):** um serviço pode ser descoberto automaticamente pela internet e contratado. É uma forma altamente evoluída de "páginas amarelas", em que serviços podem ser descobertos e contratados automaticamente por máquinas.
- **Service Delivery (Entrega de Serviço):** é a entrega efetiva do resultado da função de serviço ao cliente. Nesse caso, o cliente é qualquer usuário humano, software ou equipamento que tenha chamado o serviço. Na entrega também ocorre o fechamento do contrato de uso, envolvendo remuneração no caso de web services não gratuitos.

Web services podem desempenhar diferentes funções dependendo do contexto de interação no qual estão inseridos. O mesmo serviço pode, em diferentes momentos ou situações, assumir papéis variados ou até mesmo acumular múltiplos papéis ao mesmo tempo. Isso acontece de acordo com o cenário em que o serviço é visualizado e o estado atual da tarefa que ele está executando. Além disso, é comum

que um web service seja projetado para atuar tanto como provedor, oferecendo funcionalidades, quanto como solicitante, consumindo funcionalidades de outros serviços, como ilustrado na figura a seguir.

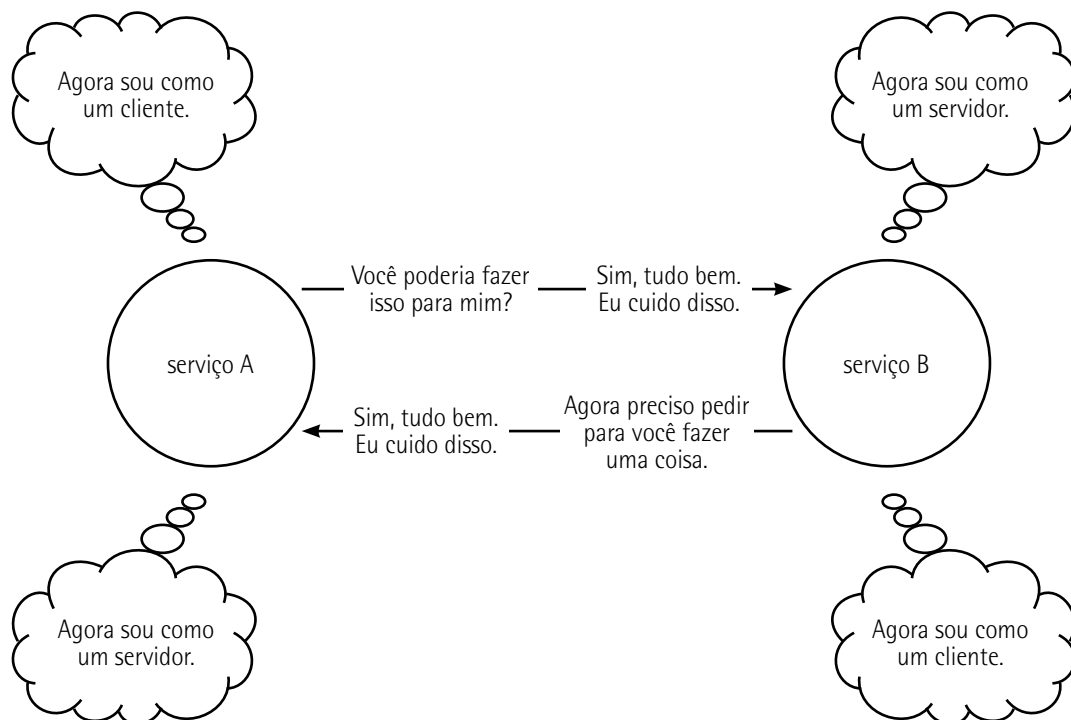


Figura 70 – Web service como provedor e solicitante

Adaptada de: Coulouris *et al.* (2013).

Os serviços também podem ser registrados em agentes de descoberta para facilitar que outras aplicações os encontrem dinamicamente.

Em um cenário típico de comunicação entre web services, o cliente que inicia uma requisição é, muitas vezes, ele próprio um web service. Isso significa que qualquer serviço que expõe uma interface para outros consumirem também pode, simultaneamente, consumir outros serviços.

Portanto, ao contrário do modelo tradicional cliente-servidor, os web services tendem a estabelecer sistemas ponto a ponto (peer-to-peer), em que qualquer serviço pode atuar tanto como cliente quanto como servidor, dependendo da situação.

Quando mensagens são trocadas entre dois ou mais web services, surgem diversos tipos de interação, e alguns conceitos são usados para melhor entender e organizar essas comunicações. Vamos explorar cada um desses conceitos:

- **Caminho da mensagem:** é o trajeto completo que a mensagem percorre desde sua origem até seu destino final. Esse caminho começa com um remetente inicial e termina em um receptor final, podendo passar ou não por vários intermediários ao longo do percurso. Esses intermediários

podem ser responsáveis por processar, modificar ou apenas redirecionar a mensagem. É importante destacar que o caminho que uma mensagem segue nem sempre é fixo, ele pode ser dinamicamente alterado por dispositivos de roteamento intermediários. A decisão sobre qual rota seguir pode ser feita levando em conta fatores como: carga de trabalho para equilibrar o tráfego entre servidores; características específicas da mensagem como seu conteúdo, destino ou prioridade; e outros critérios técnicos analisados em tempo real.

- **Padrão de troca de mensagens:** em ambientes baseados em serviços, as interações entre eles normalmente seguem padrões de comunicação pré-definidos. Esses padrões determinam como as mensagens são enviadas e recebidas. Cada padrão de troca define a dinâmica da comunicação e o tipo de relacionamento entre os serviços envolvidos. Alguns dos padrões de troca de mensagens mais conhecidos são:
 - **Pedido e resposta (request and response):** um serviço envia uma solicitação e espera uma resposta. Esse é o padrão mais comum, principalmente em interações que precisam de sincronização imediata entre as partes.
 - **Publicação e assinatura (publish and subscribe):** um serviço publica informações e outros serviços aos interessados, como se fosse um serviço para assinantes, que recebem essas informações, geralmente de forma assíncrona.
- **Correlação:** é a técnica usada para relacionar mensagens que pertencem à uma mesma interação, mas que podem ter transitado por caminhos de mensagem diferentes. Essa prática é especialmente útil no padrão de pedido e resposta, em que é fundamental vincular a resposta recebida à solicitação original que foi enviada anteriormente. Uma forma comum de realizar essa correlação é inserindo IDs (identificadores únicos) nas mensagens. Assim, mesmo que várias mensagens estejam circulando ao mesmo tempo, é possível saber qual resposta pertence a qual solicitação.
- **Coreografia:** define as regras de comportamento que regem como um grupo de web services interage entre si. Essas regras abrangem aspectos como a sequência na qual os serviços devem ser acionados, as condições que determinam o que acontece a seguir e os padrões de uso permitidos durante a interação. A coreografia seria como um "roteiro" que estabelece quem fala com quem, em que ordem e sob quais condições. Geralmente, o escopo de uma coreografia está associado a uma atividade ou tarefa específica, representando um fluxo coordenado entre múltiplos serviços.
- **Atividades:** também chamadas de tarefas, atividades são ações compostas pela colaboração de vários web services trabalhando juntos para cumprir uma função ou um conjunto lógico de funções. As atividades se baseiam nos padrões de troca de mensagens mencionados anteriormente. A diferença principal entre atividade e coreografia é que a atividade costuma estar diretamente ligada a uma função de aplicação específica. Ou seja, enquanto a coreografia é mais focada em como os serviços interagem, a atividade é mais focada em o que está sendo realizado, como a conclusão de uma tarefa de negócio.

O tipo de web service mais amplamente utilizado é o XML web service. A sigla XML significa Extensible Markup Language (Linguagem de Marcação Extensível). O XML é uma linguagem de marcação que estabelece um conjunto de regras para a estruturação e a codificação de dados, permitindo que esses dados sejam interpretados tanto por seres humanos quanto por máquinas. Essa característica torna o XML ideal para a troca de informações entre sistemas diferentes, mesmo quando desenvolvidos em plataformas distintas.

Historicamente, o XML web service se consolidou como o modelo mais amplamente utilizado e bem-sucedido no universo dos web services. Apesar da forte ascensão do JSON (especialmente para aplicações web modernas, mais leves e rápidas), o XML ainda se mantém muito presente, principalmente em ambientes corporativos. Essa permanência se deve a vários fatores, tais como:

- rigor e estrutura do XML, que ajuda a validar a integridade dos dados;
- ampla documentação e suporte consolidado em diversas plataformas;
- continuidade de sistemas legados, que já utilizam XML há anos.

Um XML web service normalmente precisa atender a dois requisitos essenciais: comunicar-se utilizando protocolos padrão da Internet, como o HTTP; e trocar informações na forma de documentos XML, formatados conforme especificações rigorosas.

O sucesso dos web services com XML estimulou o surgimento de um conjunto de tecnologias complementares que se tornaram padrões de mercado, mesmo que não formalmente obrigatórios.

Quando se cria um web service, normalmente é utilizado um documento WSDL para descrever como a interface do serviço funciona. Além disso, é comum usar o protocolo SOAP para enviar mensagens no formato XML através de HTTP. A ideia é que essas tecnologias sejam empregadas sem modificar o funcionamento essencial do serviço, mantendo sua forma padrão de comunicação. Na prática, muitas arquiteturas já partem do princípio de que o uso de SOAP e WSDL é uma premissa básica para o funcionamento de um web service.

Em 2000, o W3C (World Wide Web Consortium), organização responsável por estabelecer os padrões da Web, aceitou oficialmente a submissão do protocolo SOAP (Simple Object Access Protocol) como uma proposta de padronização para a comunicação entre sistemas distribuídos. O SOAP é um protocolo baseado em mensagens XML que criou uma estrutura padronizada para permitir a comunicação entre diferentes aplicações ou serviços através da internet, utilizando geralmente o protocolo HTTP como meio de transporte.

Essa padronização foi fundamental para dar início a uma era em que aplicações de diferentes tipos, escritas em diversas linguagens de programação, pudessem trocar informações de forma estruturada e segura.

No ano seguinte, em 2001, o W3C avançou ainda mais ao publicar a especificação do WSDL (Web Services Description Language). O WSDL é uma linguagem baseada em XML criada especificamente para descrever as interfaces dos web services. Com ela, tornou-se possível documentar exatamente como um serviço funciona, quais operações ele oferece, quais dados são esperados e quais respostas podem ser fornecidas, tudo isso de forma clara, padronizada e legível por máquinas.

Além disso, o ecossistema de web services foi enriquecido com a criação do UDDI (Universal Description, Discovery and Integration). O UDDI introduziu um mecanismo padrão para descoberta dinâmica de serviços, permitindo que aplicações pudessem encontrar automaticamente web services disponíveis na rede, sem necessidade de configuração manual.

Com SOAP, WSDL e UDDI combinados, formou-se a chamada primeira geração da plataforma de web services, estabelecendo as bases para o crescimento dessa tecnologia. Desde o surgimento desses padrões, os web services começaram a ser rapidamente adotados por fabricantes de software e empresas de tecnologia do mundo inteiro.

O forte apoio da indústria, somado à relevância dos princípios de design orientados a serviço, impulsionou não só o uso massivo desses serviços, mas também levou à criação de uma segunda geração de especificações e práticas de web services, mais maduras e especializadas.

Um web service é definido por meio de um conjunto de documentos de definição, que funcionam como blocos de construção para descrever seus serviços de forma estruturada. Essa estrutura é baseada em três principais especificações XML:

- **WSDL (Web Services Description Language):** define a interface do serviço, ou seja, quais operações estão disponíveis e como acessá-las.
- **SOAP (Simple Object Access Protocol):** define o formato das mensagens trocadas entre os serviços.
- **UDDI (Universal Description, Discovery, and Integration):** permite registrar, localizar e descrever serviços disponíveis em um diretório público ou privado.

Esses padrões, combinados com os princípios da SOA (Arquitetura Orientada a Serviços), formam a base de uma arquitetura de web services de primeira geração, construída sobre a tecnologia XML. Essa arquitetura permite o desenvolvimento de serviços independentes, capazes de encapsular funcionalidades de negócio isoladas, promovendo modularidade e reutilização.

No entanto, essa tecnologia também apresenta algumas limitações, que motivaram a criação de uma segunda geração de especificações para superar esses desafios.

A seguir, veremos com mais detalhes as tecnologias WSDL, SOAP e UDDI.

7.4.1 WSDL (Web Services Description Language)

Para que possam ser utilizados por outras aplicações e serviços, os web services precisam ser definidos de forma consistente e padronizada.

A WSDL (Web Services Description Language), uma especificação criada pelo W3C, é a linguagem mais amplamente utilizada para descrever formalmente as definições de um web service.

A camada de integração introduzida pela arquitetura de web services cria um padrão universal com interfaces programáticas compatíveis, e é justamente a WSDL que torna possível essa comunicação, fornecendo descrições padronizadas dos serviços.

A melhor forma de entender como um web service é definido por um documento WSDL é analisar seus principais elementos estruturais (os construtores que, juntos, formam a definição da interface do serviço). Os elementos principais da WSDL são:

- **Interfaces:** representam a interface do serviço e agrupam operações relacionadas. Em uma arquitetura baseada em componentes, equivalem à interface de um componente.
- **Operations:** representam funções específicas do serviço, semelhantes a métodos, e podem envolver mensagens de entrada e saída.
- **Messages:** definem os dados trocados durante uma operação, podendo conter múltiplas partes.
- **Parts:** representam os parâmetros individuais das mensagens, tanto de entrada quanto de saída.

Cada operação normalmente envolve a troca de mensagens entre o cliente (solicitante) e o provedor do serviço, seguindo o formato definido na WSDL.

Além da definição da interface abstrata, o documento WSDL também pode especificar os protocolos de comunicação que serão utilizados, como SOAP ou HTTP, completando, assim, a descrição técnica do serviço e possibilitando sua integração com diferentes plataformas.

7.4.2 SOAP (Simple Object Access Protocol)

SOAP (Simple Object Access Protocol), ou Protocolo Simples de Acesso a Objetos, é um protocolo para troca de informações estruturadas em uma plataforma descentralizada e distribuída que permite a elementos distribuídos de uma aplicação robusta se comunicarem. Sua mensagem é baseada em XML, e para a transmissão das mensagens utiliza majoritariamente o protocolo HTTP ou HTTP2.

Além disso, SOAP é uma parte integral de uma SOA (arquitetura orientada à serviços), porque permite ao remetente criar mensagens roteadas baseadas nos serviços logísticos que foram aplicados para a mensagem durante o percurso.

Uma mensagem SOAP é, de modo geral, formada pelos elementos SOAP Envelope, SOAP Header e SOAP Body.

- **Envelope:** elemento raiz da mensagem. Define um documento XML como uma mensagem SOAP, o que torna obrigatório que um documento XML seja uma mensagem SOAP formatada de acordo com o padrão estabelecido.
- **Header:** elemento opcional no XML, mas quando utilizado, é o primeiro elemento-filho do elemento Envelope. Nesse elemento, podem-se encontrar informações extras que precisam ser passadas ao servidor, como informações de segurança do WS-Security.
- **Body:** elemento obrigatório que contém os dados de negócio que o servidor está esperando.

7.4.3 UDDI (Universal Description, Discovery, and Integration)

UDDI (Universal Description, Discovery, and Integration), ou Descrição, Descoberta e Integração Universal, é como uma lista telefônica da internet para serviços.

A especificação UDDI introduziu um mecanismo padronizado para a descoberta dinâmica de serviços, permitindo que aplicações localizassem e acessassem descrições de web services de forma automatizada. Com isso, o UDDI ajudou a estabelecer a primeira geração das plataformas de web services.

O funcionamento é como o das antigas páginas amarelas, nas quais era possível encontrar os dados de um restaurante, por exemplo, com nome, endereço, telefone e tipo de serviço. Ou seja, uma lista pública que todos podem acessar e encontrar os serviços requeridos.

O UDDI faz isso, só que para os web services. Os web services ficam disponíveis como programas na internet para que outras empresas ou aplicativos possam usar. Por exemplo, um serviço que calcula frete de entrega ou um que converte moedas. Ele funciona como um "catálogo online", no qual serviços da internet são cadastrados, pesquisados e usados, para facilitar que programas de empresas diferentes se encontrem e conversem automaticamente.

O UDDI serve para cadastrar serviços (como publicar o que o serviço faz, como usá-lo e onde encontrá-lo) e também para encontrar serviços, buscando por tipo de serviço ou nome, por exemplo. Isso facilita a integração entre sistemas e empresas diferentes pela internet.

7.5 SOA (Service Oriented Architecture)

SOA (Service Oriented Architecture), ou Arquitetura Orientada a Serviços, é um conceito de arquitetura que busca disponibilizar as funcionalidades de um sistema como um serviço. Dessa forma, essas funcionalidades podem ser compartilhadas e reutilizadas entre aplicações. Seu principal objetivo é ser mais flexível em atender às necessidades do mercado.

Como já mencionado, integrar uma aplicação com alguns web services é algo simples e, em muitos casos, útil. Esse tipo de integração limitada pode ser apropriado para fins de aprendizado ou para complementar uma aplicação existente, adicionando uma funcionalidade pontual baseada em serviços que atenda a uma necessidade específica do projeto.

No entanto, esse tipo de uso não caracteriza uma SOA. Existe uma diferença fundamental entre uma aplicação que apenas consome web services e uma aplicação construída com base nos princípios de SOA.

A SOA é um modelo de arquitetura cujo conceito central está ligado ao encapsulamento da lógica de negócio em serviços bem definidos. Nesse modelo, os web services se tornam os blocos de construção principais, formando um novo paradigma de desenvolvimento. Ao adotar uma arquitetura SOA, a aplicação assume um compromisso com os princípios de projeto orientado a serviços, bem como com as tecnologias que os suportam, como XML, WSDL, SOAP e UDDI.

Uma SOA baseada em web services XML se apoia em camadas padronizadas de tecnologia, com foco em expor funcionalidades da aplicação como serviços fracamente acoplados. Para viabilizar esse modelo, a arquitetura geralmente conta com mecanismos de descoberta de serviços, como service brokers ou agentes de descoberta.

É importante destacar que a adoção da SOA traz uma complexidade maior ao projeto. Mais do que em uma arquitetura multicamadas tradicional, os arquitetos e desenvolvedores precisam analisar cuidadosamente como a introdução de serviços impactará dados existentes, processos de negócio e integração entre sistemas.

À medida que o uso de serviços se expande, os requisitos de segurança, desempenho e escalabilidade se tornam ainda mais críticos. Arquiteturas SOA bem projetadas enfrentam esses desafios com o apoio de uma infraestrutura robusta e padronizada, evitando soluções customizadas que dependam de uma única aplicação.

A SOA não diz respeito a detalhes de implementação. Nesse ponto, muitos dos detalhes de como implementar uma Arquitetura Orientada a Serviços se confundem com técnicas de pura integração de sistemas. No entanto, várias das técnicas aplicáveis à integração de sistemas não são aplicáveis em SOA, o que faz com que ambas apenas compartilhem algumas características em comum, sem serem equivalentes.

Um dos componentes mais importantes da SOA é o ESB (Enterprise Service Bus), ou Barramento de Serviços Corporativos.

O ESB (veja a figura a seguir) é um modelo de arquitetura de software usado para projetar e implementar a comunicação entre aplicativos de software que interagem mutuamente em uma arquitetura SOA. Como um modelo de arquitetura de software para computação distribuída, é uma variante especial do modelo cliente-servidor mais geral e promove agilidade e flexibilidade em

relação à comunicação entre aplicativos. Seu uso principal é na integração de aplicativos corporativos de paisagens heterogêneas e complexas.

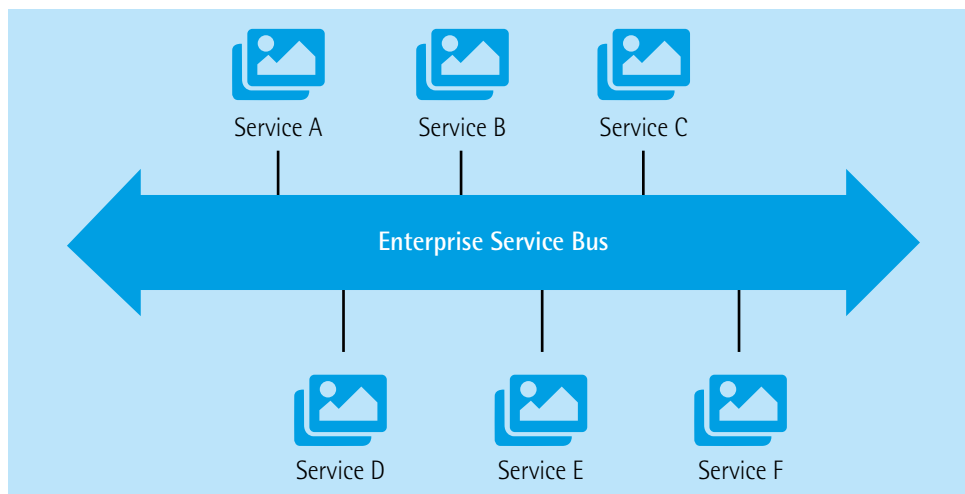


Figura 71 – ESB (Enterprise Service Bus)

Esse barramento disponibiliza com maior facilidade os serviços do sistema para os usuários e outras aplicações, acelerando processos de integração. Um ESB transporta o conceito de design de sistemas operacionais modernos para redes de computadores díspares e independentes.

As principais vantagens da SOA são:

- alinhamento com as regras de negócio;
- diminuição do tempo de desenvolvimento;
- facilidade de manutenção devido ao baixo acoplamento entre as partes do sistema;
- flexibilidade durante mudanças, beneficiadas pelo isolamento da estrutura de um serviço;
- facilidade de agregar novas tecnologias a plataformas;
- possibilidade de reutilização de componentes.

Para implementar a SOA com sucesso é importante ter infraestrutura bem desenvolvida, arquitetura de referência consolidada e uma política de governança para direcionar os trabalhos.

Na literatura científica, a IoS (Internet dos Serviços) é mencionada como um dos pilares da Indústria 4.0, juntamente com a IoT (Internet das Coisas) e os CPS (sistemas ciberfísicos) (Kagermann; Wahlster; Helbig, 2013).

O termo Internet de Serviços surgiu da convergência de outros dois conceitos: Web 2.0 e SOA. Nesse contexto, SOA é uma forma de projetar e construir um conjunto de aplicações em que componentes e serviços web disponibilizam suas funções no mesmo canal de acesso para uso mútuo.

Através do conceito SOA, novas aplicações podem ser montadas a partir dos componentes e serviços disponíveis, como um Lego®. Assim, todas as aplicações de uma organização podem oferecer e consumir serviços em um canal de comunicação único e integrado, denominado Enterprise Service Bus (ESB), como uma forma simples de facilitar a integração.

O foco da SOA é alavancar a criação de blocos funcionais reutilizáveis e interoperáveis para reduzir a quantidade de esforços de reprogramação. Metaforicamente, seria como o subsolo de um shopping center, no qual diversos serviços, como barbearia, conserto de celulares e alfaiataria, são oferecidos em um mesmo local físico, facilitando o acesso do cliente.

Em um sistema de manufatura em nuvem, vários recursos e habilidades de manufatura podem ser detectados de forma inteligente e conectados à internet mais ampla por meio dos princípios da SOA. Como resultado, temos os SOMS (Sistemas de Manufatura Orientada a Serviços), e é dessa fusão de conceitos que surge a Internet de Serviços como um dos componentes fundamentais da Indústria 4.0.

Dessa forma, vemos que vários assuntos que parecem desconexos em um primeiro momento, no final estão todos interconectados. Além disso, nada precisa ser inventado do zero, pelo contrário, conceitos já existentes e interações entre os recursos e serviços de sistemas criados previamente fornecem estrutura para novos sistemas e soluções.



Saiba mais

Para explorar mais esse assunto, leia o artigo:

REIS, J. Z.; GONÇALVES, R. F. The role of Internet of Services (IoS) on industry 4.0 through the Service Oriented Architecture (SOA). In: MOON, I. et al. *Advances in Production Management Systems. Smart Manufacturing for Industry 4.0*. Seul: Springer Cham, 2018. p. 20-26.

7.6 Virtualização

A virtualização é uma evolução tecnológica que permite transformar recursos que antes estavam ligados diretamente a um hardware específico em serviços de TI mais flexíveis e eficientes. Essa técnica possibilita o aproveitamento máximo da capacidade de uma máquina física, dividindo seus recursos de forma inteligente entre diferentes usuários ou ambientes.

Para exemplificar, consideremos que uma empresa possua três servidores físicos: um para gerenciar e-mails, outro para hospedagem de sites e um terceiro que roda sistemas antigos, ainda essenciais para as operações internas. Embora cada servidor esteja em uso, eles operam com apenas cerca de 30%

de sua capacidade. Mesmo assim, é necessário mantê-los funcionando, especialmente por conta das aplicações legadas que não podem ser desativadas.

Esse modelo convencional, no qual cada servidor é dedicado a uma única tarefa, era comum por questões de segurança e facilidade na administração. Cada servidor rodava um sistema operacional com uma função específica. No entanto, isso levava a um grande desperdício de recursos. Com a virtualização, esse paradigma mudou. Agora, é possível dividir um único servidor físico em várias máquinas virtuais independentes, cada uma atuando como se fosse um servidor completo.

Com essa nova abordagem, por exemplo, é possível dividir o servidor de e-mail em duas máquinas virtuais, cada uma executando tarefas distintas de forma isolada. Dessa maneira, os sistemas legados podem ser transferidos para uma dessas máquinas virtuais, eliminando a necessidade de manter um equipamento físico exclusivamente para isso. O mesmo hardware é utilizado de forma muito mais eficiente.

Além disso, levando em conta fatores como segurança e desempenho, o servidor pode ser ainda mais segmentado para abrigar novas funções, elevando seu aproveitamento de 30% para 60%, e depois para até 90%. Os equipamentos que estavam sendo pouco utilizados ou até mesmo sem uso podem ser reaproveitados em outras funções ou simplesmente desligados, reduzindo os custos com energia, refrigeração e manutenção.

8 APLICAÇÕES DE SISTEMAS DISTRIBUÍDOS MODERNOS

Aplicações de sistemas distribuídos modernos são compostas tanto por computadores de alta performance quanto por pequenos dispositivos móveis, trabalhando de forma independente, mas não isolada, em favor da colaboração, um dos princípios fundamentais para esse tipo de mecanismo funcionar.

Explorar os sistemas distribuídos modernos pode ser uma jornada desafiadora, repleta de complexidades e surpresas. Em um mundo cada vez mais conectado e digital, essas aplicações se tornaram a base de serviços críticos que sustentam setores como streaming de mídia, e-commerce, bancos digitais, redes sociais, computação em nuvem e IoT.

Sistemas distribuídos são compostos por múltiplos componentes independentes que cooperam entre si, muitas vezes espalhados geograficamente, para oferecer serviços integrados, escaláveis e resilientes. Eles trazem benefícios essenciais para as organizações que operam aplicações críticas, como alta disponibilidade, capacidade de escalar conforme a demanda, flexibilidade operacional e resiliência diante de falhas. Em um mercado altamente competitivo, dominar os sistemas distribuídos pode ser um grande diferencial para liderar a inovação frente aos competidores.

No entanto, esse ecossistema é um ambiente intrinsecamente complexo, no qual fatores como múltiplos nós de comunicação, latência de rede, problemas de consistência de dados e interdependência de componentes criam um cenário propenso a falhas e a desafios imprevisíveis.

Assim, fazer as escolhas corretas de tecnologias e práticas torna-se algo absolutamente crucial. Elementos como rastreamento detalhado do fluxo de requisições, registro rigoroso de eventos, execução eficiente em *containers* usando ferramentas como *dockers* e *kubernetes*, estratégias sólidas de DevOps, processos bem definidos de integração e entrega contínuas, além de um desenho arquitetural inteligente e robusto, são apenas algumas das peças fundamentais para garantir a estabilidade, observabilidade e eficiência dos sistemas distribuídos modernos.

É vital reconhecer que as chances de falha são altas. Pequenos erros de decisão na escolha de tecnologias, na definição da arquitetura ou na operacionalização dos serviços podem rapidamente comprometer todo o sistema, levando a problemas graves como instabilidade, perda de dados, interrupções de serviço e aumento dos custos operacionais.

Nesse cenário, não existe receita pronta ou um script que pode ser seguido em qualquer situação ou problema. Cada ambiente distribuído carrega suas próprias particularidades, exigindo adaptação constante, monitoramento contínuo e resiliência técnica e organizacional. Para trabalhar com sistemas distribuídos é necessário estar preparado para enfrentar condições adversas, antecipar riscos e agir rapidamente para evitar impactos maiores.

Entretanto, é justamente dentro dessa complexidade que se encontra o potencial transformador dos sistemas distribuídos. Quando bem projetadas e gerenciadas, essas aplicações proporcionam inovação acelerada, resiliência a falhas em larga escala, escalabilidade sob demanda e vantagem competitiva sustentável. Esses são diferenciais cada vez mais críticos para negócios que almejam crescimento, inovação contínua e liderança de mercado.

Em suma, o sucesso em sistemas distribuídos não depende apenas da tecnologia, mas também da capacidade de se compreender os riscos, adotar as melhores práticas, aprender com os erros e, sobretudo, transformar os desafios em diferenciais estratégicos. Dominar os sistemas distribuídos modernos não é apenas um requisito técnico, mas uma estratégia de sobrevivência e crescimento para as organizações do futuro. Dada a importância dos sistemas distribuídos modernos, vejamos algumas aplicações e ferramentas utilizadas.

8.1 Microsserviços

Microsserviços são uma forma específica de construir aplicações em que cada parte do sistema é desenvolvida como um serviço independente. Cada um desses serviços pode ser instalado, atualizado e ampliado de forma separada, sem depender diretamente da aplicação completa.

Em uma arquitetura mais tradicional, ou monolítica, toda a aplicação é construída como um grande bloco único, dividido internamente apenas em bibliotecas e componentes que funcionam dentro do mesmo ambiente. Já na arquitetura de microsserviços, cada módulo funciona como um pequeno programa separado. Ele recebe solicitações, realiza o processamento necessário e devolve uma resposta ao solicitante, normalmente utilizando a comunicação pela internet, através do protocolo HTTP.

Essa maneira de desenvolver sistemas permite que cada serviço evolua, escale e seja mantido de maneira independente, trazendo mais flexibilidade, agilidade e resistência a falhas para o sistema como um todo.

Em sistemas distribuídos modernos, a arquitetura de microsserviços quebra uma aplicação grande em vários serviços pequenos, independentes e especializados, que se comunicam entre si, normalmente via APIs. Cada microsserviço é responsável por uma única função de negócio.

A ideia não é exatamente nova, já que vem sendo usada em ambientes Unix desde a década de 1960. No entanto, recentemente, a prática de sistemas distribuídos tornou-se o epicentro de uma grande revolução na forma como as empresas desenvolvem softwares ágeis, a partir de equipes enxutas responsáveis por componentes autossuficientes.

Para entender melhor a motivação por trás de microsserviços, vale relembrar que os projetos são programados, geralmente, em três grandes partes: o lado "cliente", com a interface do usuário; uma base de dados, com as informações do sistema; e uma camada "servidor", com a lógica da aplicação. A camada "servidor" lida com as requisições do usuário, executa a lógica de negócio, retorna e atualiza dados da base e disponibiliza informações prontas para o cliente exibir. Essa camada é um monólito ou aplicação monolítica, uma vez que gera uma única e grande aplicação com tudo junto. Qualquer mudança no sistema envolve compilar tudo novamente e implantar uma nova versão da camada "servidor".

Microsserviços, na forma conceitual, são aplicações práticas que partem do princípio de decomposição funcional, tanto ao dividir as capacidades de negócio quanto ao permitir mais agilidade para as equipes de desenvolvimento. O conceito que eles representam é muito semelhante ao da SOA, que já vimos nesta unidade. Na prática, microsserviços são uma evolução natural de SOA. Devido às semelhanças entre os dois conceitos, é comum que SOA e microsserviços sejam confundidos. No entanto, a principal diferença entre eles está no escopo.

A SOA é uma abordagem arquitetural mais ampla, adotada em nível corporativo, com o objetivo de integrar diferentes sistemas e áreas da empresa por meio de serviços reutilizáveis. Já os microsserviços compõem uma estratégia de implementação aplicada pelas equipes de desenvolvimento, geralmente no contexto de uma única aplicação, dividindo-a em pequenos serviços independentes e especializados.

O conceito de microsserviços é fundamental nas tecnologias modernas de cloud computing e virtualização, além de serem utilizados também em componentes como o middleware.

Outra diferença importante está na forma de comunicação entre os componentes. Na SOA, normalmente é utilizado um ESB (Enterprise Service Bus), que centraliza e gerencia a troca de mensagens entre serviços. Já nos microsserviços, a comunicação é feita de forma stateless (sem manutenção de estado entre as chamadas), por meio de APIs independentes de linguagem, geralmente usando HTTP/REST ou mensagens assíncronas com filas.

Essa independência das APIs nos microsserviços permite que as equipes escolham as linguagens, frameworks e ferramentas com que se sentem mais confortáveis, tornando os microsserviços uma abordagem mais flexível e tolerante a mudanças em comparação com a SOA tradicional.

É importante entender que trabalhar com microsserviços vai muito além de simplesmente rodar partes de um sistema em containers. Quando os componentes começam a crescer desordenadamente e surgem problemas de dependência entre eles, isso indica que os microsserviços não estão sendo bem aplicados. Ou seja, o simples uso de containers não garante a adoção correta dessa arquitetura.

Embora nem todo sistema distribuído seja composto de microsserviços, podemos afirmar que estes representam uma categoria especial dentro do universo dos sistemas distribuídos. Eles são fortemente relacionados à infraestrutura de redes e camadas de comunicação, como a chamada "malha de serviços", sempre mantendo limites transacionais bem definidos entre diferentes contextos de negócio.

Com o tempo, um serviço pode se tornar tão complexo que o código pode ficar difícil de entender e manter. Quando isso acontece, o desenvolvimento de novas funcionalidades e a correção de erros passam a ser tarefas cada vez mais complicadas.



Observação

À medida que microsserviços se tornam mais numerosos e complexos, a gestão da comunicação entre eles pode se tornar um desafio. É aí que entra a malha de serviços, uma camada de infraestrutura dedicada a gerenciar a comunicação, segurança, roteamento e observabilidade entre microsserviços. Ferramentas como Istio e Linkerd permitem implementar esses recursos de forma padronizada, sem que cada time de desenvolvimento precise reescrevê-los em seus serviços.

Na evolução em direção aos microsserviços, a arquitetura SOA foi fundamental durante quase vinte anos, estabelecendo conceitos que ainda são seguidos. A principal diferença entre SOA e microsserviços não está no conceito, mas sim na infraestrutura e na agilidade no processo de implantação dos serviços. Esse avanço só foi possível graças às inovações tecnológicas que surgiram ao longo dos anos, como os containers (dockers) e as plataformas de orquestração (kubernetes), que facilitaram muito o trabalho com aplicações distribuídas.

Antigamente, o processo de implantação de componentes em SOA era mais rígido, demorado e dependente, visto que ferramentas de automação e containerização ainda eram raras ou inexistentes. Com a evolução tecnológica, microsserviços ganharam espaço justamente pela facilidade de gerenciamento, atualização e escalabilidade.

Resumidamente, microsserviços são sistemas distribuídos, por isso carregam consigo todas as vantagens e também todos os desafios desse tipo de arquitetura, sendo os principais: a definição correta

dos limites entre os serviços; a escolha adequada do nível de separação (granularidade); e a necessidade de lidar com a consistência eventual dos dados.

Muitos sistemas distribuídos bem-sucedidos começaram suas jornadas como aplicações monolíticas e, à medida que amadureceram, foram divididos em serviços menores, autônomos e independentes. Dessa forma, a transição de um monólito para microsserviços deve ser feita com bastante planejamento e cuidado. Um monólito bem estruturado, ao ser desmembrado, tem muito mais chances de se transformar em um sistema distribuído eficiente e de alta performance.

8.1.1 Containers

Container é algo que contém coisas, um local para armazená-las e eventualmente transportá-las. No âmbito dos microsserviços, o container armazena e garante que o microsserviço tenha tudo que ele precisa, como o código da aplicação, bibliotecas e configurações, ou seja, tudo que é necessário para o microsserviço funcionar de maneira isolada do restante do sistema.

Portanto, na programação, podemos chamar de containers todos os objetos escalares, logo, os objetos que são compostos por outros objetos. São estruturas de dados diversas que carregam objetos ou outras estruturas, podendo conter, inclusive, outros containers. No entanto, é necessário lembrar que um objeto escalar é o próprio objeto e não contém nada a não ser a si próprio.

Para facilitar o gerenciamento das aplicações distribuídas em um sistema complexo, o conceito de middleware layer pode ser aplicado, atuando como uma espécie de container, oferecendo uma camada de abstração para a infraestrutura e hardware, facilitando a intercomunicação entre as aplicações pertencentes ao mesmo cluster de execução. Estruturas e classes de objetos podem ser considerados containers porque possuem um ou mais dados dentro dele. Os containers mais óbvios são as coleções, como o array, que possui uma sequência de dados.

8.1.2 Dockers

Dockers são uma forma de empacotar os microsserviços, isto é, a tecnologia para criar e executar containers. O Docker agrupa partes de softwares de um sistema de arquivo completo e que abrange todos os recursos necessários para a sua execução, logo, ele é uma plataforma de containers.

Por ser uma ferramenta que cria e gerencia containers, o docker facilita o processo de empacotar uma aplicação dentro de um container.

Antes da criação dos dockers, montar ambientes de sistema distribuído era muito trabalhoso, pois dependia da configuração manual de servidores, instalar programas, acertar versões etc. Com o docker, o ambiente é construído uma vez e pode rodar em qualquer lugar com apenas alguns comandos. É como criar um molde para rodar a aplicação de forma rápida.

8.1.3 Kubernetes

Kubernetes é uma plataforma de orquestração para gerenciar os múltiplos containers. Ele facilita muito a criação, o empacotamento e a distribuição dos microsserviços. Isso significa que tudo pode ser instalado no servidor e armazenado nos containers. Dessa forma, os mesmos softwares e as suas versões podem ter uma execução facilitada em qualquer ambiente de desenvolvimento.

Além disso, o kubernetes organizam e administram vários containers ao mesmo tempo. Quando se tem apenas um ou dois containers, é fácil controlar. Mas em sistemas grandes, pode haver centenas ou milhares de containers funcionando juntos. Os itens empacotados, ou até mesmo um ambiente inteiro, são dispostos nos containers e se tornam portáteis, o que torna o trabalho conjunto mais eficiente. Ao mesmo tempo, a implantação pode ser feita em ambientes não heterogêneos.

Portanto, kubernetes são como uma plataforma inteligente que cuida para que todos os containers estejam sempre funcionando do melhor jeito possível.

8.1.4 API

API (Application Programming Interface), ou Interface de Programação de Aplicativos, é um conjunto de rotinas e padrões de programação para acesso a um aplicativo de software ou plataforma baseado na web. APIs são o meio de comunicação entre os microsserviços. Cada microsserviço expõe uma API (normalmente REST) para que outros serviços possam interagir com ele de forma padronizada.

APIs existem para software, hardware e até sistemas operacionais. Dependendo do estilo de arquitetura escolhido para organizar os componentes de uma aplicação, é comum usar o conceito de **ports and adapters**, ou portas e adaptadores. Essa abordagem sugere que a aplicação disponibilize interfaces de entrada para receber e processar solicitações vindas de outros sistemas, por meio de mecanismos de comunicação entre processos, como o envio de mensagens pela rede.

Ao mesmo tempo, a aplicação também utiliza interfaces de saída, que permitem que ela acesse recursos externos, como bancos de dados, servidores de e-mail, arquivos e outros serviços necessários durante a execução de suas atividades de negócio.

Dentro desse modelo, o núcleo da aplicação, onde fica a lógica principal, se conecta apenas a essas interfaces, chamadas de portas, sem se preocupar com os detalhes de como a comunicação ou o acesso a recursos externos é implementado. Esses detalhes ficam por conta dos adaptadores, que são responsáveis por traduzir as chamadas da aplicação para a tecnologia necessária em cada caso.

8.1.5 API Gateway

O API Gateway é um tipo de suporte que se destaca por proporcionar segurança aos usuários. Na prática, ele atua como um filtro de entrada que possui o intuito de direcionar os dados ao ponto mais adequado possível.

O Gateway possui diferentes aplicabilidades, como:

- **Filtrar o tráfego no momento da integração dos meios:** mobile, web, sistemas cloud, entre outros.
- **Porta de entrada única:** o Gateway é a única porta de entrada para os APIs e para a organização deles.
- **Roteador:** o sistema também funciona como um roteador de tráfego dos APIs.
- **Seguro:** garante proteção por possuir acesso autenticado e limitação de conexão dos usuários.

A variedade de funções do Gateway o torna significativamente importante, uma vez que, além de automatizar os serviços, ele mantém o controle de tudo em somente um local na web. Esse local atua como uma base de dados que concentra todas as informações que se deseja.

8.1.6 REST

REST (REpresentational State Transfer), ou Transferência de Estado Representativo, é um estilo de desenvolvimento de web services. De forma prática, REST é uma maneira organizada e padronizada de usar o protocolo HTTP para que diferentes sistemas possam conversar entre si.

Em um sistema REST, são usadas as URLs para identificar recursos, como usuários, produtos e pedidos. Além disso, são usados verbos HTTP para dizer o que se quer fazer, como:

- **GET:** buscar dados.
- **POST:** criar dados.
- **PUT:** atualizar dados.
- **DELETE:** apagar dados.

As informações geralmente são trocadas em JSON, um formato leve e fácil de ler. Tais práticas direcionam não só o uso adequado dos métodos HTTP, mas também o uso adequado de URLs (Universal Resource Locator, ou Localizador Uniforme de Recursos) e URIs (Universal Resource Identifier, ou Identificador Uniforme de Recursos).

Uma URI pode ser utilizada para identificar qualquer coisa ou dar um caminho para um determinado conteúdo, nome etc., enquanto que uma URL pode ser utilizada apenas para fornecer caminhos. Assim, uma URL é uma forma de URI.

8.2 Blockchain

O Blockchain é um sistema distribuído para aplicação em cadeias de valor que exigem o registro de dados operacionais ou financeiros. Ele funciona como um livro-razão compartilhado em um sistema distribuído, que possui a função de registrar transações, executar contratos eletrônicos e também realizar o rastreamento de ativos.

Durante o seu funcionamento, ocorre o armazenamento de blocos, que possui informações sobre uma aplicação específica, permitindo identificar dispositivos inteligentes e programá-los conforme as interações baseadas nas codificações de regras preestabelecidas. Tais regras, que o blockchain é capaz de armazenar e validar, são informações de domínios específicos nos blocos chamadas de smart contracts, ou contratos inteligentes.

A figura a seguir ilustra o armazenamento de registros devidamente criptografados em blocos em operação em um sistema computacional distribuído.

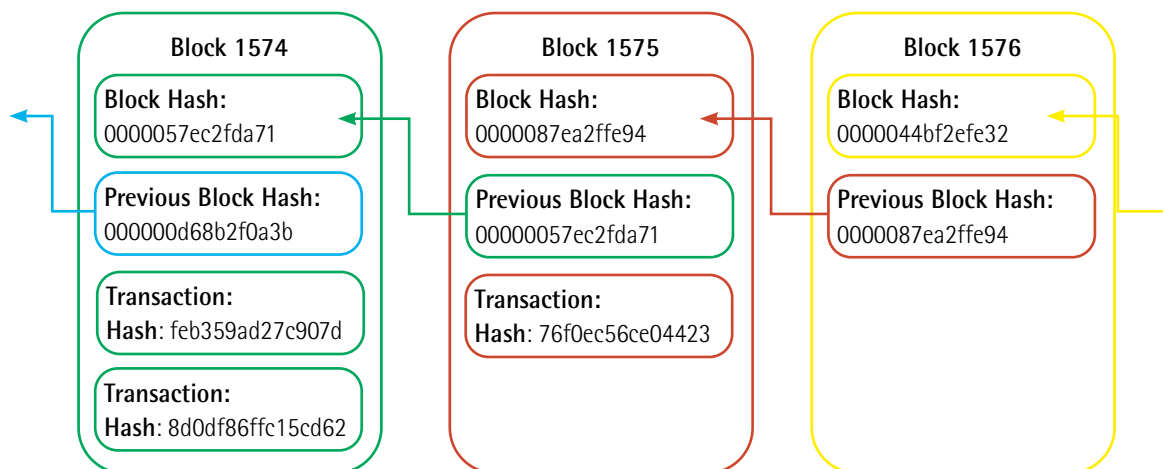


Figura 72 – Blockchain em série de blocos conectados

Adaptada de: Mougayar (2017).

O blockchain é uma arquitetura de blocos de dados vinculados por criptografia. Os blocos são organizados em uma sequência temporal e validados por um algoritmo com funções hash. Funções hash são pequenos códigos de computador que convertem qualquer classe de dados em uma sequência de tamanho fixo, independentemente do tamanho dos dados de entrada.

Dessa forma, o blockchain é uma estrutura de banco de dados composta por endereços combinados para equilibrar valores, com cópias distribuídas em todos os nós de uma rede. Cada bloco no blockchain contém todos os dados transacionais do sistema desde o seu início e a impressão digital única usada para confirmar as informações. Os blocos são conectados linear e cronologicamente com o próximo bloco por meio do hash, como uma cadeia real. A criação de um novo bloco depende de provas de trabalho, que são problemas matemáticos que exigem poder computacional para serem resolvidos.

O blockchain pode ser considerado como parte banco de dados, parte plataforma de desenvolvimento, parte viabilizador de rede. Não há entidade central controladora e nenhum participante é mais confiável do que outro. Existe, portanto, um consenso descentralizado, em que cada participante realiza a validação das regras existentes (Mougayar, 2017).

A primeira aplicação do blockchain foi a criptomoeda Bitcoin. O blockchain também pode ser usado para registrar, rastrear e controlar transferências de ativos como um sistema de inventário. Portanto, qualquer ativo tangível ou digital poderia ser armazenado no blockchain.

De acordo com o funcionamento do blockchain, a possibilidade de armazenamento de registros com a implementação de contratos inteligentes para transações em domínios distintos se tornou viável. Uma das áreas possíveis de implementação, e que está se configurando de maneira promissora no que diz respeito à validação e controle, é a própria internet. Como o blockchain está em uma camada acima da internet, ele pode ter muitas formas de implementação.

O blockchain funciona na internet como um banco de dados compartilhado e sincronizado entre vários computadores, sem uma autoridade central. Devido a isso, foi sugerido que a automação com blockchain poderia ser viável em fluxos de trabalho que demandam tempo considerável para serem executados.

Na indústria, a aplicação de blockchain é vista como uma alternativa para aumentar a automação de processos e reduzir os custos transacionais. As análises comparativas de desempenho entre cenários sem e com blockchain são pertinentes para avaliar sua adoção.



Lembrete

O blockchain funciona como um livro-razão distribuído e compartilhado entre diversos computadores conectados em rede. Sua principal finalidade é registrar transações de forma segura, executar contratos inteligentes, ou smart contracts, e possibilitar o rastreamento de ativos digitais ou físicos.

O blockchain pode ser configurado em dois tipos de arquitetura: pública ou privada. No blockchain público, todos os usuários podem ler, enviar transações e cooperar na prova de trabalho sem qualquer autorização, de forma que todas as transações são públicas e os usuários permanecem anônimos. Por outro lado, o blockchain privado é executado por uma única entidade que autoriza o acesso a usuários selecionados.

A característica central da validação de blocos é a execução da prova de trabalho, que consiste na resolução de um quebra-cabeça de hash. Somente blocos que contenham uma solução precisa de um único hash serão processados. Essa confirmação é uma resposta a um problema matemático que utiliza funções de hash unidirecionais. De forma simplificada, o problema consiste em determinar uma variável que produza um resultado cujo início seja uma sequência específica pré-determinada, quando combinada com dados do bloco anterior e do bloco atual e processada por uma função de hash. Por

exemplo, o esforço poderia ser descobrir um valor de string cujo resultado comece com três zeros à esquerda. Como essa função é unidirecional, essa resolução precisa ser feita por tentativa e erro, e à medida que a rede se expande, o sistema blockchain calibra a dificuldade de encontrar a resposta para o problema aumentando o número de zeros.

São várias as áreas com potencial aplicação de blockchain: redes de logística e suprimentos, indústrias, sistema de aluguel de veículos autônomos, contrato de câmbio, automação de royalties, licenciamento de direitos autorais, uso de criptomoeda social para iniciativas sociais e ambientais, entre muitas outras.

8.3 Aplicações de IoT

IoT (Internet of Things), ou Internet das Coisas, foi um termo criado em 1999 para descrever um sistema no qual o mundo material se comunica com a internet através da troca de dados com sensores onnipresentes. Uma década depois de sua criação, entre 2008 e 2009, o número de dispositivos conectados à rede excedeu o número de habitantes do nosso planeta. Esse momento, de acordo com a Cisco, é o verdadeiro nascimento da Internet das Coisas (Witkowski, 2017).

A IoT é um paradigma no qual uma grande quantidade de objetos inteligentes conversa com a internet através de sensores e atuadores. Esses dispositivos de comunicação, que coletam e transmitem dados através de uma rede, geralmente estão espalhados geograficamente e se comunicam para realizar tarefas específicas, tornando-a um sistema distribuído por natureza.

O acesso mais facilitado à internet permitiu um enorme salto em produtividade, visto que a IoT é a primeira revolução de infraestrutura inteligente da história, e está conectando cada equipamento, empresa, indústria, residência, veículo em uma rede inteligente.

Com o advento da IoT, objetos como máquinas, veículos, eletrodomésticos e outros são conectados diariamente, de forma que possam ser acessados remotamente por dispositivos móveis como celulares e notebooks ou por dispositivos fixos como computadores desktop desde que estejam conectados à internet.

Além do grande impacto econômico que a IoT irá gerar, ela afetará também a sociedade e as organizações, criando novas oportunidades aos que se anteciparem aos seus movimentos.

Com a IoT, várias redes concorrentes são implantadas e personalizadas para fornecer serviços. Alguns desenvolvimentos desse novo paradigma são: o surgimento de mundos virtuais; ambientes sensoriais; interações entre os mundos físico e virtual; sistemas embarcados e comunicações veiculares; e desenvolvimentos na vida digital com todos os aplicativos relacionados para auxiliar o bem-estar dos cidadãos.

A integração da computação em nuvem com aplicações de IoT representa outro grande salto. O uso das tecnologias de nuvem oferece vários benefícios, como: elasticidade por meio do acionamento automático de serviços e infraestruturas; redução de custos, já que a infraestrutura e o tamanho das

plataformas podem ser adaptados às demandas de serviço; modelos pay-per-use, que permitem que apenas o consumo real de recursos seja pago; e maior disponibilidade dos serviços pela replicação de componentes. As aplicações decorrentes dessa integração abrem novas direções para negócios, mas apresentam também desafios em relação ao ambiente de computação e comunicação nos aspectos de escalabilidade, interoperabilidade, mobilidade, adaptabilidade, privacidade e segurança.

Cidades inteligentes utilizam tecnologias para soluções em mobilidade urbana, segurança civil, otimização de recursos como energia e água, além de ferramentas como smart grids, entre outras. A tecnologia passa, assim, a ser essencial como a água, o gás e a eletricidade. Seja para revitalizar uma cidade ou construir uma nova, a tecnologia estará ali inserida. Tais estratégias passam pela premissa de uma conexão cada vez maior, não somente entre pessoas, mas também entre inúmeros dispositivos, sensores e máquinas à internet por meio da IoT.

A pluralidade de serviços possíveis originados dessa maior conectividade provê um ambiente de soluções no qual vendedores e consumidores podem interagir entre si pela internet, e isso torna possível um gerenciamento e uma coordenação autônomos. Tudo passa a ser oferecido como serviço na internet, incluindo aplicações, a plataforma para entregá-las, além da própria infraestrutura de rede.

O alcance global da inovação e da abordagem digital, com o advento da internet das coisas e serviços, permite que as empresas já nasçam globais. Como exemplificam Coviello e Tanev (2017), se compararmos empresas desenvolvedoras tradicionais de software com empresas de SaaS (Software-as-a-Service), vemos que ambas atuam no setor de software, mas as SaaS apresentam potencial de já nascerem globais. Isso porque o que elas oferecem já está totalmente digitalizado, e basta o clicar de um botão para se comunicarem e alcançarem seu mercado mundo afora.

Uma plataforma de serviços sobre IoT segue uma arquitetura de três camadas, composta por uma camada de dispositivos de IoT, uma camada de gateways de IoT e uma camada de servidores e aplicações, conforme ilustrado na figura a seguir, que traz um modelo de arquitetura IoT.

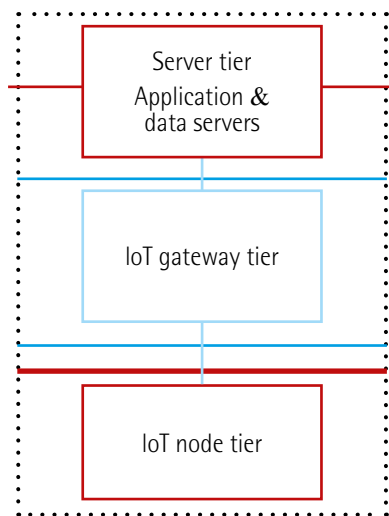


Figura 73 – Arquitetura em Camadas do IoT

Adaptada de: Sanchez *et al.* (2014).

O envolvimento de usuários finais concretos acrescenta outra dimensão às capacidades de avaliação da plataforma, permitindo não apenas a avaliação do desempenho técnico das soluções de IoT, mas também sua adoção pelos usuários e seu impacto social.

A camada de nós ou dispositivos da IoT consiste nos sensores de IoT espalhados por toda uma localidade. A camada de gateway de IoT conecta os dispositivos nas bordas da rede por meio de Wi-Fi, 3G, anel de fibra óptica etc. A camada de servidores fornece dispositivos de servidores mais potentes, diretamente conectados à infraestrutura da rede central. Os servidores hospedam repositórios de dados e servidores de aplicativos que podem ser configurados para habilitar uma variedade de serviços e aplicativos diferentes ou para investigar abordagens para mineração de dados e engenharia do conhecimento no mundo real. A camada de servidores se beneficia da virtualização em uma infraestrutura de nuvem, garantindo alta confiabilidade e disponibilidade de todos os componentes e serviços, diferentemente dos dispositivos na primeira camada de nós, que podem ser expostos às condições ambientais e correm risco de danos por vandalismo, insalubridade, temperatura, umidade etc.

Observa-se também na figura anterior que as implantações de IoT são essencialmente soluções fechadas e verticalmente integradas, adaptadas a domínios de aplicação específicos. Ao atingir a terceira camada, onde a IoT e a inteligência da arquitetura ocorrem, os principais serviços de habilitação permitem a interoperabilidade dos recursos de IoT implantados em diferentes aplicativos e contribuem para uma reutilização horizontal da infraestrutura. Essa abordagem pode ser explorada em várias aplicações. Alguns exemplos são de aplicações em cidades inteligentes, tais como:

- **Monitoramento do ambiente:** as soluções que já existiam para monitoramento do ambiente nos centros urbanos geralmente dependiam de um pequeno número de estações de medição instaladas em locais fixos. Embora a precisão dos equipamentos de medição nessas unidades seja alta, seu custo efetivamente exclui a implantação em larga escala para obter medições com granularidade mais fina. Com a introdução da tecnologia IoT, um grande número de sensores de baixo custo pode ser implantado usando um grande número de pontos de medição, o que torna possível a realização de medições precisas. As leituras coletadas por sensores fixos e móveis podem ser usadas, por exemplo, como indicador inicial da gravidade da poluição ambiental em grandes áreas. Caso sejam observadas condições adversas, alarmes especiais são gerados pelo sistema. Informações sobre os bairros com maior nível de poluição do ar podem ser disponibilizadas posteriormente para outros acessos de diferentes serviços.
- **Gestão de estacionamentos:** por meio de sensores ferromagnéticos sem fio enterrados sob o asfalto, esse serviço de cidade inteligente permite o monitoramento da ocupação de vagas de estacionamento externo nas ruas da cidade para fins de uso e contabilização. Equipamentos de rede, como repetidores, são implantados em uma área para garantir a conectividade com a internet, de modo que as informações sobre a ocupação do estacionamento possam ser disseminadas instantaneamente aos motoristas, aos órgãos de controle de tráfego competentes da cidade ou às autoridades locais. Além disso, os dados dos sensores das vagas de estacionamento são agregados e usados para alimentar painéis de informações sobre o status do estacionamento em cruzamentos. Esses dados podem ser acessados por aplicativos de celular, fornecendo, por exemplo, auxílio de navegação para liberar vagas de estacionamento.

Da mesma forma, os dados históricos de ocupação do estacionamento podem ser analisados pelas autoridades municipais para determinar o nível de oferta de vagas na cidade, melhorando a logística e a segurança.

- **Realidade aumentada:** o caso de uso de realidade aumentada visa a ampliar a paisagem urbana ou locais na cidade com endpoints de IoT para fornecer informações e serviços contextuais nesses locais. Isso envolve inicialmente o aumento de POIs (Points of Interest), ou Pontos de Interesse, na cidade, como pontos turísticos, lojas e espaços públicos. A realidade aumentada é usada para expor serviços ou informações relevantes sobre o local ou contexto aos visitantes. Como exemplo desse serviço, a tela do celular do visitante no local pode ser sobreposta com serviços relevantes ou informações direcionadas, dependendo da localização ou direção de visão do turista. Outro exemplo do uso da realidade aumentada, é a forma que ela proporciona uma experiência de passeio, fornecendo informações sensíveis à localização, como a descrição de monumentos no idioma preferido do turista. Oportunidades de vendas também podem ser anunciadas aos clientes, proporcionando às lojas novas oportunidades para construir e fortalecer relacionamentos. Informações de localização e frequência de visitantes podem ser usadas para avaliar a popularidade dos locais e ajustar as estratégias de marketing ou de governança por vários setores, comerciais ou sociais.

A IoT (Internet das Coisas) tem possibilitado cenários e soluções que, até pouco tempo atrás, estavam restritos a grandes corporações ou exigiam investimentos significativos em infraestrutura, algo fora do alcance da maioria das empresas.

Embora o termo **Internet das Coisas** seja relativamente novo, a ideia de conectar dispositivos a datacenters ou sistemas distribuídos já existe há décadas, com aplicações em áreas como caixas eletrônicos, controle remoto de subestações elétricas e monitoramento em tempo real de frotas de ônibus. No entanto, esses projetos exigiam altos investimentos, apresentavam riscos consideráveis e demandavam um longo período para serem implementados.

Estima-se que até 2030 existam mais de 1 trilhão de dispositivos IoT em operação. A antiga premissa de que apenas computadores e notebooks poderiam se conectar à internet tornou-se ultrapassada. Atualmente, dispositivos conectados estão em todos os lugares, e a expectativa é de que seu número continue crescendo exponencialmente. Esse crescimento é impulsionado por fatores como a redução no custo de sensores, a maior disseminação do acesso à internet, aumento do poder de processamento e diminuição dos custos de coleta e de armazenamento de dados graças às soluções em nuvem (Duraes; Ferreira; Manzan, 2022).

As projeções de mercado para o setor costumam apresentar números impressionantes, que estimulam o interesse de empresas e impulsionam investimentos em novas tecnologias, criação de empregos e surgimento de oportunidades diversas.

Com a popularização da IoT e o crescimento da cultura maker, muitos passaram a acreditar que bastam um dispositivo de hardware, como o Arduino, e um serviço online gratuito para desenvolver soluções complexas. De fato, surgiram inúmeras startups nos últimos anos que combinam hardware,

software e conectividade para entregar soluções inovadoras baseadas em IoT (Duraes; Ferreira; Manzan, 2022).

No entanto, apesar do movimento de democratização dos recursos computacionais, desenvolver soluções profissionais em IoT está longe de ser uma tarefa simples. A criação de sistemas bem estruturados exige conhecimento em diversas áreas, como sensores, consumo de energia, protocolos de comunicação e serviços em nuvem, tornando esse processo um verdadeiro desafio técnico.



Saiba mais

Para saber mais sobre soluções IoT, leia:

DURAES, W.; FERREIRA, F. H. I. B.; MANZAN, R. *Arquitetura de soluções IoT: desenvolva com Internet das Coisas para o mundo real*. São Paulo: Casa do Código, 2022.



Resumo

Esta unidade apresentou um estudo completo sobre sistemas distribuídos, começando pela sua definição como um conjunto de computadores independentes que operam de forma cooperativa para parecer um único sistema aos usuários. Essa estrutura surgiu com o avanço da computação, quando se percebeu a necessidade de compartilhar recursos e integrar informações espalhadas em diferentes locais físicos. Diferentemente dos sistemas centralizados do passado, os sistemas distribuídos permitem que o processamento seja descentralizado, proporcionando maior flexibilidade, escalabilidade e acessibilidade aos serviços, mesmo que os componentes estejam geograficamente distantes.

As principais características desses sistemas incluem a transparência para o usuário, a comunicação por mensagens entre componentes de hardware e software e a capacidade de operar com falhas parciais sem comprometer o todo. Exemplos práticos como a internet, intranets corporativas, computação em nuvem e aplicativos móveis demonstram como os sistemas distribuídos fazem parte do cotidiano. Esses sistemas permitem que diferentes tecnologias operem em conjunto, ocultando a complexidade de sua infraestrutura para oferecer uma experiência integrada e eficiente.

Diversos desafios acompanham os sistemas distribuídos, como garantir uma comunicação confiável, coordenação entre processos, tolerância a falhas, segurança e desempenho. O texto explorou aspectos técnicos como o uso de protocolos (TCP/IP, DNS, TLS), modelos de consistência de dados (forte, sequencial, eventual) e ferramentas como message brokers para garantir a integridade e a ordem das mensagens. Também foi discutido a importância de estratégias como replicação de dados, cache e uso de edge computing para reduzir latência e melhorar a performance.

A escalabilidade é tratada como um dos principais benefícios e, ao mesmo tempo, um grande desafio. Pode-se escalar verticalmente (reforçando um único nó) ou horizontalmente (adicionando novos nós), usando estratégias como duplicação, decomposição funcional e particionamento de dados. O uso de microserviços surge como uma evolução natural da SOA (Arquitetura Orientada a Serviços), permitindo que aplicações sejam divididas em serviços menores e independentes, mais fáceis de manter e escalar.

Outro tema central é a tolerância a falhas, que exige mecanismos como verificação de saúde dos componentes (health checks), isolamento (bulkhead), interrupção temporária de conexões falhas (circuit breaker) e uso de padrões como retry, rate limiting e load leveling. Esses elementos são cruciais para manter a resiliência do sistema mesmo diante de sobrecargas ou falhas de rede. O desempenho, por sua vez, depende do equilíbrio entre latência, throughput, eficiência do código, proximidade dos dados e arquitetura de comunicação assíncrona.

No que se refere à segurança, a unidade destacou a necessidade de criptografia em trânsito e em repouso, autenticação forte, autorização baseada em privilégios mínimos e auditoria contínua das ações realizadas no sistema. Além disso, são utilizadas práticas como rotação de segredos, segmentação de redes e uso de cofres seguros para proteger dados sensíveis. A segurança em sistemas distribuídos deve ser contínua, acompanhando a evolução das ameaças e incorporando-se ao ciclo de vida completo do sistema, desde a concepção até a operação.



Exercícios

Questão 1. Um sistema distribuído é um conjunto de computadores independentes entre si, que se comunicam por meio de uma rede de comunicação de dados e que se apresentam aos usuários como um sistema único e coerente.

Nesse contexto, assinale a alternativa que corresponde a um sistema distribuído:

- A) Um único computador pessoal executando um programa offline.
- B) Um mainframe centralizado com terminais burros.
- C) A internet, que conecta usuários e serviços em máquinas dispersas pelo mundo.
- D) Uma calculadora de mesa que não se conecta a outros dispositivos.
- E) Um videogame que funciona sem conexão à internet.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: um computador offline é um sistema isolado (não distribuído). Sistemas distribuídos requerem múltiplos computadores interconectados.

B) Alternativa incorreta.

Justificativa: mainframes com terminais burros são exemplos de sistemas centralizados, um modelo antigo que foi gradualmente substituído por sistemas distribuídos.

C) Alternativa correta.

Justificativa: a internet é considerada o maior exemplo de sistema distribuído por conectar máquinas e serviços globalmente.

D) Alternativa incorreta.

Justificativa: calculadoras isoladas nem mesmo se conectam a redes.

E) Alternativa incorreta.

Justificativa: videogames offline são sistemas autônomos. Aplicativos de jogos que se conectam a servidores são exemplos de sistemas distribuídos, mas não são sistemas desconectados.

Questão 2. (IADES/2019, adaptada) A computação em nuvem é uma área recente da computação que trabalha com a transferência de informação e o acesso de arquivos a distância. Com base no que foi estudado, avalie as afirmativas:

I – Existem empresas especializadas em disponibilizar serviços em nuvem.

II – A computação em nuvem permite o armazenamento de dados com capacidade infinita, sem alterar o valor cobrado pelo serviço.

III – Não existem softwares para serem instalados localmente, tendo em vista que a computação em nuvem visa à descentralização geográfica.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e III, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa A.

Análise das afirmativas

I – Afirmativa correta.

Justificativa: há diversas empresas especializadas em fornecer serviços de computação em nuvem. A Amazon oferece o AWS (Amazon Web Services), a Google oferece o Google Cloud e a Microsoft oferece o Azure, por exemplo.

II – Afirmativa incorreta.

Justificativa: a computação em nuvem entrega recursos de TI sob demanda por meio da internet, com preço definido pela utilização. Desse modo, o valor cobrado pelo serviço é uma função da utilização demandada. Além disso, embora a computação em nuvem permita grande capacidade de armazenamento de dados, essa capacidade não é infinita.

III – Afirmativa incorreta.

Justificativa: embora a computação em nuvem permita o acesso a aplicativos e a dados de qualquer lugar, alguns softwares podem precisar ser instalados localmente para acessar certos serviços em nuvem.

REFERÊNCIAS

- APPIAH-OTOO, I.; SONG, N. The impact of ICT on economic growth-comparing rich and poor countries. *Telecommunications Policy*, v. 45, n. 2, p. 102082, mar. 2021.
- CISCO. *Cisco Certified Network Associate CCNA 200-301: Official Cert Guide*. Indianapolis: Cisco Press, 2021.
- CISCO. *Designing for Cisco Internetwork Solutions: Student Guide*. San Jose: Cisco Systems, 2010.
- CISCO. *Interconnecting Cisco Network Devices 1.0a: Student Guide*. San Jose: Cisco Systems, 2000.
- COMER, D. *Redes de computadores e a internet*. Porto Alegre: Pearson Education do Brasil, 2016.
- COULOURIS, G. et al. *Sistemas distribuídos: conceitos e projeto*. São Paulo: Bookman, 2013.
- COVIELLO N.; TANEV, S. Initiating a new research phase in the field of international entrepreneurship: an interview with professor Nicole Coviello. *Technology Management Review*, v. 7, n. 5, p. 52-56, 2017.
- DURAES, W.; FERREIRA, F. H. I. B.; MANZAN, R. *Arquitetura de soluções IoT: desenvolva com Internet das Coisas para o mundo real*. São Paulo: Casa do Código, 2022.
- HELERBROCK, R. Rede 5G. *Mundo Educação*, [s.d.]. Disponível em: <https://tinyurl.com/559ezkdc>. Acesso em: 17 jun. 2025.
- KAGERMANN, H.; WAHLSTER, H.; HELBIG, J. Securing the future of German manufacturing industry: recommendations for implementing the strategic initiative INDUSTRIE 4.0 – Final Report of the Industrie 4.0 working group. *Acatech – National Academy of Science and Engineering*, p. 1-82, abr. 2013.
- KUNTZ, M. PaaS vs. ESB: Which integration approach is best for your organization. *See Burger Blog*, 2 jun. 2021. Disponível em: <https://tinyurl.com/4m2hvdas>. Acesso em: 17 jun. 2025.
- KUROSE, J. F.; ROSS, K. W. *Redes de computadores e a internet: uma abordagem top-down*. 8. ed. São Paulo: Pearson Education do Brasil, 2021.
- LEZOCHÉ, M. et al. Agri-food 4.0: a survey of the supply chains and technologies for the future agriculture. *Computers in Industry*, v. 117, 2020.
- MAIA, L. P. *Arquitetura de redes de computadores*. Rio de Janeiro: LTC, 2017.
- MOUGAYAR, W. *Blockchain para negócios: promessa, prática e aplicação da nova tecnologia da internet*. Rio de Janeiro: Alta Books, 2017.
- MULTIREDE. *TCP/IP Básico*. São Paulo: Multirede Informática Ltda., 2000.

OLIVEIRA, K.; REHDER, W. *Redes de computadores*. São Paulo: Viena, 2006.

PARVIAINEN, P.; TIHINEN, M. Tackling the digitalization challenge: how to benefit from digitalization in practice. *IJISPM – International Journal of Information Systems and Project Management*, n. 5, v. 1, p. 63-77, 2017.

REIS, F. Escopos de rede – Curso de Redes de Computadores. *Bóson Treinamentos*, 22 jun. 2016. Disponível em: <https://tinyurl.com/2ubk2j94>. Acesso em: 17 jun. de 2025.

REIS, J. Z.; GONÇALVES, R. F. The role of Internet of Services (IoS) on industry 4.0 through the Service Oriented Architecture (SOA). In: MOON, I. et al. *Advances in Production Management Systems. Smart Manufacturing for Industry 4.0*. Seul: Springer Cham, 2018. p. 20-26.

SAARIKKO, T.; WESTERGREN, U. H.; BLOMQUIST, T. Digital transformation: five recommendations for the digitally conscious firm. *Business Horizons*, p. S0007681320300975, ago. 2020.

SACOMANO J. B. et al. *Indústria 4.0*. São Paulo: Blucher, 2018.

SANCHEZ, L. et al. SmartSantander: IoT Experimentation over a Smart City Testbed. *Computer Networks*, p. 217-238, mar. 2014.

SHANNON; C. E. A mathematical theory of communication. *Bell System Technical Journal*, n. 5, p. 379-423, 1948.

SILVA, M. T. et al. *Transformação digital e Indústria 4.0: produção e sociedade*. São Paulo: Blucher, 2023.

TANENBAUM, A.; FEAMSTER, N.; WETHERALL, D. *Redes de computadores*. 5. ed. Rio de Janeiro: Pearson Prentice Hall, 2021.

TANENBAUM, A. S.; STEEN, M. *Sistemas distribuídos: princípios e paradigmas*. São Paulo: Pearson, 2007.

UNIDIR. *ITU-R Update: IMT-2030 (6G)*. Genebra, 2024. Disponível em: <https://tinyurl.com/bdhd8t7e>. Acesso em: 26 maio 2025.

W3C. *Web Services Architecture*. Massachusetts, 2004. Disponível em: <https://tinyurl.com/2wzbu28d>. Acesso em: 26 maio 2025.

WITKOWSKI, K. Internet of Things, Big Data, Industry 4.0: innovative solutions in logistics and Supply Chains Management. *Procedia Engineering*, v. 182, p. 763-769, 2017.



A series of horizontal lines for writing, consisting of 28 evenly spaced lines across the page.



Informações:
www.sepi.unip.br ou 0800 010 9000